

Introduction and the Machine Learning Workflow — Commentary

Lesson 1 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

Not examinable · reading time about 2 hours

Contents

Slide 1 — Lesson 1: Introduction and the ML Workflow	3
Slide 2 — Welcome	4
Slide 3 — What we will cover	5
Slide 4 — What this course is not	6
Slide 5 — Your advantage, and your gap	7
Slide 6 — How you will be assessed	8
Slide 7 — The three things you get each lesson	9
Slide 8 — Getting the material	10
Slide 9 — Today	11
Slide 10 — A problem you cannot specify	12
Slide 11 — Every rule buys an exception	13
Slide 12 — The inversion	14
Slide 13 — What the data actually is	15
Slide 14 — Making it precise	16
Slide 15 — The setup, in one picture	17
Slide 16 — What we actually want	18
Slide 17 — What we can actually compute	19
Slide 18 — Accuracy: the share of right answers	20
Slide 19 — Learning, in one line	21
Slide 20 — Learning is a search	22
Slide 21 — The gap that explains everything	24
Slide 22 — The lowest training error is the worst model	26
Slide 23 — Which gives us the one rule for today	27
Question — who is p?	28
Slide 24 — When <i>not</i> to use machine learning	30
Slide 25 — Seventy years in one picture	31
Slide 26 — 1943: the first artificial neuron	32
Slide 27 — 1950: Turing's imitation game	33
Slide 28 — 1956: Dartmouth	34
Slide 29 — 1957: the perceptron learns	35
Slide 30 — 1969: the first winter	37
Slide 31 — 1986: backpropagation	39
Slide 32 — 1995-2010: statistics takes over	40

Slide 33 — 2012: AlexNet	42
Slide 34 — The pattern	43
Slide 35 — Three kinds of learning	44
Slide 36 — Supervised	45
Slide 37 — Unsupervised	46
Slide 38 — Self-supervised	47
Slide 39 — One dataset, three questions	48
Slide 40 — Where the target comes from	49
Slide 41 — Notebook 1, live	50
Slide 42 — The workflow	51
Slide 43 — Notebook 2, live	52
Slide 44 — Step 1: frame it	53
Slide 45 — Which error is worse?	54
Slide 46 — Step 2: look first	55
Slide 47 — Step 3: split, before anything else	56
Question — what is a single test score worth?	56
Slide 48 — Step 4: baseline first	61
Slide 49 — Step 5: pipeline, not two steps	62
Slide 50 — What the pipeline prevents	63
Slide 51 — Step 6: the result	64
Slide 52 — What accuracy hides	65
Slide 53 — Precision and recall	67
Slide 54 — Precision and recall, counted	68
Slide 55 — Behind every label, a probability	69
Slide 56 — The trade-off, drawn	70
Slide 57 — What we did not do	72
Slide 58 — Four ways a model misleads you	73
Slide 59 — Notebook 3, live	74
Slide 60 — 200 samples. 5000 random features. Coin-flip labels.	75
Slide 61 — Select features first, split second	76
Slide 62 — The rule	78
Slide 63 — 99% accuracy, detecting nothing	79
Slide 64 — Two models, one accuracy	80
Slide 65 — Shortcut features	82
Slide 66 — Does this value exist yet?	83
Slide 67 — One split is one measurement	84
Slide 68 — What the four have in common	85
Slide 69 — A model is not objective	86
Slide 70 — Homework: we discuss it on Friday 9 October	87
Slide 71 — Before next week	88
The thread of the whole lesson	89

What this document is. A companion to the slides of lesson 1: for every slide, what it means, the steps it leaves implicit, and the questions it tends to raise, answered under the slide they belong to. It is a second explanation of the same material, in a different voice from the handout, which remains the reference for the derivations. It is **not examinable**: nothing here is needed for the exam that is not already in the handout and the notebooks.

Slide numbers and titles are the ones in the lesson's slide deck, in **Slides/**. Where a section mentions “the notes”, it means the speaker notes stored in the slide deck, which PowerPoint shows below each slide.

Slide 1 — Lesson 1: Introduction and the ML Workflow

It is simply the cover, but it already establishes two important things: this is the **introductory lesson**, and its subject is not “a first ML algorithm” but the **workflow**.

It is a significant choice. A course could start directly with linear regression; here the aim is to teach first **how a machine learning experiment must be built**.

This anticipates a philosophy that will come back again and again:

a correct algorithm placed inside a wrong experiment still produces a wrong conclusion.

Slide 2 — Welcome

The slide immediately makes the level clear:

A first course in machine learning, for people who can already program.

So it does not assume that you know ML, but it does assume they can already think algorithmically and write code.

The phrase “**Foundations, not products**” is fundamental. It means the course does not want to teach how to use the AI product of the moment, but the principles that remain valid when libraries and architectures change.

The notes contain an interesting observation: those who have already trained a few models start with an advantage, but they may also have some bad habits, above all in **evaluation**. It is very realistic: it is relatively easy to learn

```
model.fit(X_train, y_train)
model.score(X_test, y_test)
```

and much harder to understand whether `X_test` is really independent of everything done before.

This slide, then, sets out the course’s contract:

programming the model will be the relatively easy part; showing that the result is credible will be the hard part.

Slide 3 — What we will cover

The structure of the course is:

1–2 workflow and data preparation,
3–4 regression and classification,
5 experimental methodology,
6–7 k-NN, SVMs, trees and ensembles,
8 unsupervised learning,
9–10 neural networks and CNNs.

The most interesting choice is **Lesson 5: Experimental Methodology**, deliberately placed in the middle.

In many courses experimental methodology comes at the end, after the student has already spent weeks comparing accuracies and models. Here it comes before the systematic comparison between model families.

So the progression is essentially:

learn a model $\rightarrow$ learn to evaluate it $\rightarrow$ compare models.

The remark about LLMs also makes sense: they are not part of the course, but the concepts studied here are part of their foundations. A Transformer is still a parametric model optimised against a loss on data, so problems such as generalisation, data distribution and validation continue to exist.

Slide 4 — What this course is not

Three negations:

- Not a tour of current AI products
- Not a library tutorial
- Not a course where you call `.fit()` and report the number

The third is the one that really matters.

The problem is not obtaining:

```
accuracy = 0.94
```

but knowing:

1. on which data it was computed;
2. how the model and hyperparameters were chosen;
3. whether the test set influenced anything;
4. what uncertainty comes with the number;
5. whether that metric measures what actually matters.

The message of the notes is very strong:

learning the scikit-learn application programming interface (API) takes little; learning **which number to trust** takes much longer.

This will probably be one of the threads running through the whole course.

Slide 5 — Your advantage, and your gap

Here the audience is characterised very well.

The **advantage** is the ability to program. You will be able to implement methods, not just use them.

The **gap**, instead, is:

you can produce results faster than you can judge them.

It is almost a definition of the modern problem of applied ML.

Writing 30 lines that produce 95% accuracy can be very easy. Establishing whether that 95% generalises requires reasoning about distribution, split, leakage, metrics and uncertainty.

The sentence in the notes is very effective:

the most common failure is not a bug: the code works exactly as written, but the number it produced was not valid.

This distinction matters for experienced programmers, because we are normally used to thinking:

correct program $\Rightarrow$ correct result.

In ML this does not hold.

We can have:

correct code + flawed experiment $\Rightarrow$ wrong conclusion.

Slide 6 — How you will be assessed

The exam format directly reinforces the course's philosophy.

The weekly exercises are not handed in. At the exam one of the ten notebooks is drawn and the student must discuss **their own work**.

The interesting consequence is that simply memorising a result becomes difficult. The questions will inevitably be of the kind:

Why did you split here?

Why did you choose this metric?

Why did you scale the features?

What would change with a different threshold?

So theory and practice are not examined separately.

The notes also specify that **methodological correctness** weighs more than raw accuracy. This is exactly in line with the course: a 95% model obtained correctly can be better, from the exam's point of view, than a 99% one obtained by contaminating the test set.

Slide 7 — The three things you get each lesson

The division is:

Handout → theory and mathematics

Slides → reasoning and intuition

Notebooks → implementation

The notes make explicit that the mathematical derivations are not removed: they are simply not loaded onto the slides.

This makes it possible to keep, in the lecture, a sequence of the kind:

problem → intuition → result → experiment

while the full formalisation stays in the handout.

It avoids turning every lesson into three hours of algebra without losing rigour: the algebra is in the handout, to be read after the lecture.

Slide 8 — Getting the material

Here the technical infrastructure is introduced:

```
git clone ...  
docker compose pull  
docker compose up
```

and then JupyterLab at:

```
127.0.0.1:8888
```

The important architectural idea is the separation between: Docker image = software environment
and

Git repository = course content.

So weekly updates normally require only a `git pull`, not the download of a whole new image.

The separation between the **core** image, sufficient up to lesson 8, and **full**, which adds TensorFlow for the lessons on neural networks, is also very sensible.

This slide contains no ML, but it removes an enormous source of noise: version differences, missing packages and local configurations.

Slide 9 — Today

This slide makes the arc of the lesson explicit:

what learning from data means → a short history → kinds of learning → workflow → ways a model can mislead you.

The last point is deliberately the most important.

The first part provides language and intuition; the last answers the question:

even if the model works and the number looks good, how can it be wrong?

From this point on the lesson actually enters machine learning.

Slide 10 — A problem you cannot specify

Example: building a spam filter.

The fundamental idea is not that writing the program is hard.

The problem is:

we do not know how to write the rule.

For example:

```
if "buy now" in mail:  
    spam
```

fails on a legitimate newsletter.

A blacklist of senders fails when the sender changes or a legitimate account is compromised.

So the problem is not computational. We could implement any rule we were given.

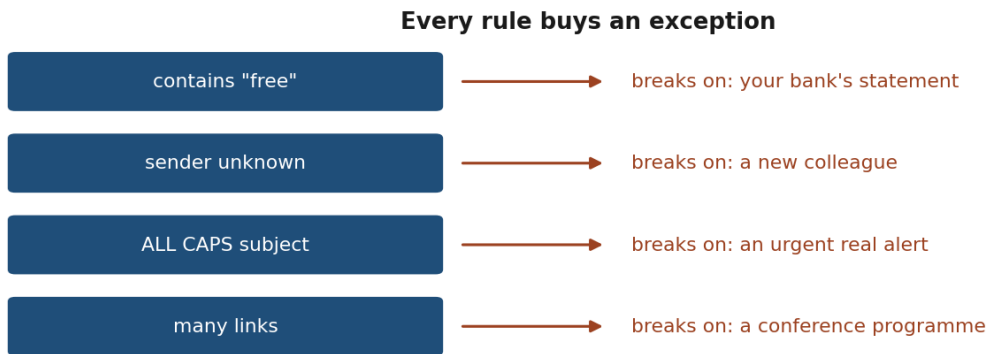
The problem is that we cannot build an explicit symbolic function:

$$f(\text{email}) = \begin{cases} 1 & \text{spam} \\ 0 & \text{legitimate} \end{cases}$$

by writing all its conditions ourselves.

This is a very clean motivation for machine learning.

Slide 11 — Every rule buys an exception



you cannot state the rule, but you recognise the answer

The problem you cannot specify. Every rule catches some spam and some legitimate mail, and the list never converges.

The figure makes the previous problem concrete:

contains "free"	→	breaks on a bank statement
sender unknown	→	breaks on a new colleague
ALL CAPS	→	breaks on a genuine urgent alert
many links	→	breaks on a conference programme

The final sentence sums it all up:

you cannot state the rule, but you recognise the answer

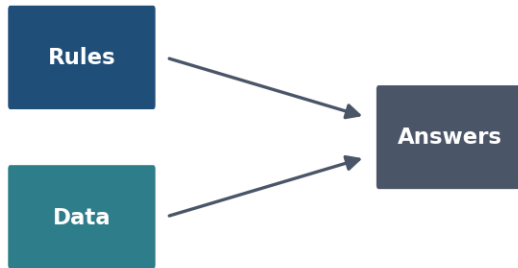
A human being can look at many cases and classify them well without necessarily being able to formalise the procedure.

This creates the space for machine learning: examples → learned regularity.

The important thing is that we are not saying “rules do not exist”. We are saying that we do not have an operational specification complete enough to program them by hand.

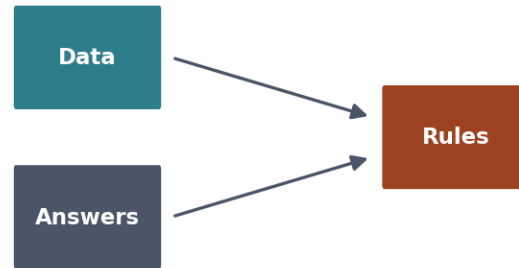
Slide 12 — The inversion

Traditional programming



you write the rules

Machine learning



the rules are learned

The inversion. Programming takes rules and data and produces answers; learning takes data and answers and produces the rules.

This is one of the most important conceptual slides.

In traditional software:

$$\text{Rules} + \text{Data} \rightarrow \text{Answers}$$

In supervised machine learning:

$$\text{Data} + \text{Answers} \rightarrow \text{Rules}$$

Of course “rules” must be read broadly: it does not necessarily mean readable symbolic rules. They could be coefficients, the weights of a network, support vectors, the splits of a tree, and so on.

The inversion, then, is:

traditional programming: we write the function;

ML: we choose a class of functions and use data to determine one of them.

In more mathematical terms: $\mathcal{F} = \{f_\theta\}$

is a family of possible functions, and learning consists of finding a parameter θ that makes f_θ fit the data.

What the lesson will later call **empirical risk minimisation** is the formalisation of this idea.

Slide 13 — What the data actually is

An excellent slide for introducing notation without starting immediately from abstract symbols.

A tumour is represented by 30 numbers:

$$x = (x_1, x_2, \dots, x_{30}).$$

For example:

$$x_1 = 17.99, \quad x_2 = 10.38, \quad x_3 = 122.80, \dots$$

That whole vector is **one example**.

The label is: $y = \text{malignant}$

possibly coded as a number.

With 569 tumours we have: $m = 569$

examples and

$$n = 30$$

features.

The design matrix is therefore: $X \in \mathbb{R}^{569 \times 30}$.

More generally:

$$X = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_m^T \end{bmatrix} \in \mathbb{R}^{m \times n}.$$

The last observation in the notes is also very important:

the 30 columns **are not the tumour**.

They are a representation of the tumour chosen by someone.

This implicitly introduces the problem of features: what the model can learn is limited to what we have made observable to it.

Slide 14 — Making it precise

Here the abstract formulation finally arrives.

The inputs belong to a space: $x \in \mathcal{X}$

and the outputs: $y \in \mathcal{Y}$.

The pairs: (x, y)

are generated according to an unknown distribution: $(x, y) \sim \mathcal{D}$.

We are looking for a function: $f : \mathcal{X} \rightarrow \mathcal{Y}$.

If the prediction is: $\hat{y} = f(x)$,

the loss function: $L(\hat{y}, y)$

measures how wrong that prediction is.

Why does $\mathcal{D}$ appear? Because we want to model not only the dataset we hold, but the population of cases we could observe.

The dataset is a finite collection of samples from that distribution.

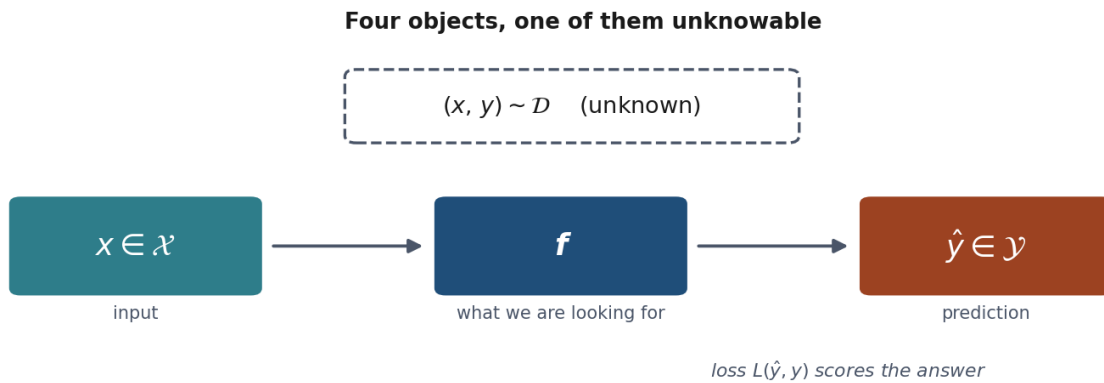
A very important implicit assumption It is normally assumed that training and future deployment come from the same distribution:

$$\mathcal{D}_{\text{train}} \approx \mathcal{D}_{\text{future}}.$$

If the hospital, sensor, population or production process changes, this equality can fail.

This is the problem of **distribution shift**.

Slide 15 — The setup, in one picture



The setup in one picture: an unknown distribution, a sample drawn from it, a model chosen using that sample, and the world it will actually meet.

The diagram sums up the entities just defined: $(x, y) \sim \mathcal{D}$,

then: $x \xrightarrow{f} \hat{y}$

and finally: $L(\hat{y}, y)$.

The conceptually most important point is the dashed box containing $\mathcal{D}$.

We observe: $(x_1, y_1), \dots, (x_m, y_m)$,

but we never directly observe the distribution $\mathcal{D}$.

And this is where almost the whole problem of generalisation comes from.

We can measure the model on finite examples, but what we really care about is its behaviour on the process that will generate future examples.

Slide 16 — What we actually want

Now the **true risk**, or **expected risk**, appears:

$$R(f) = \mathbb{E}_{(x,y) \sim \mathcal{D}} [L(f(x), y)].$$

It is probably the most important formula in the first part of the lesson.

What does it mean concretely? Imagine, ideally, being able to take infinitely many new patients from the same population.

For each one we compute: $L(f(x), y)$.

Then we take the average.

That average is $R(f)$.

If it were classification with the zero-one loss:

$$L(\hat{y}, y) = \begin{cases} 0 & \hat{y} = y \\ 1 & \hat{y} \neq y, \end{cases}$$

then: $R(f) = P(f(X) \neq Y)$.

That is, the probability that the model is wrong on a new example.

But there is the fundamental problem $\mathcal{D}$ is unknown.

So:

$R(f)$ cannot be computed directly

and this is where the need for statistics comes from.

Slide 17 — What we can actually compute

What we can compute instead is the **empirical risk**:

$$\hat{R}_S(f) = \frac{1}{m} \sum_{i=1}^m L(f(x_i), y_i).$$

It is simply the average loss over the available dataset.

With the zero-one loss it becomes:

$$\hat{R}_S(f) = \frac{\text{number of errors}}{m}.$$

So:

$$\text{accuracy} = 1 - \hat{R}_S(f).$$

The example in the notes is:

143 test examples, 2 errors.

Then: $\hat{R} = \frac{2}{143} \approx 0.01399$.

So:

$$\text{accuracy} = 1 - \frac{2}{143} = 0.9860.$$

that is, about: 98.6%.

Empirical Risk Minimisation The principle becomes:

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \hat{R}_S(f).$$

So we search the family $\mathcal{F}$ for the model that minimises the empirical loss.

Many of the course's algorithms can be read precisely in this way.

Slide 18 — Accuracy: the share of right answers

The word has been used since the first slide; this slide pins it down, on the formula of the previous one. Take the simplest loss there is, the **zero-one loss**: 1 for a wrong answer, 0 for a right one. The average loss over a sample is then just the fraction of wrong answers, the **error rate**, and

$$\text{accuracy} = \frac{\text{right answers}}{\text{all answers}} = 1 - \text{error rate}.$$

Every accuracy in the lesson is an empirical risk under a friendlier name. Notebook 02's model gets 141 of the 143 test tumours right: accuracy $141/143 = 0.986$, error rate $2/143 = 0.014$.

The slide's last bullet is a warning to come back to: the zero-one loss charges a malignant tumour sent home exactly what it charges a benign tumour sent to biopsy. That is what the confusion matrix of slide 52 exposes, and the subject of lesson 4.

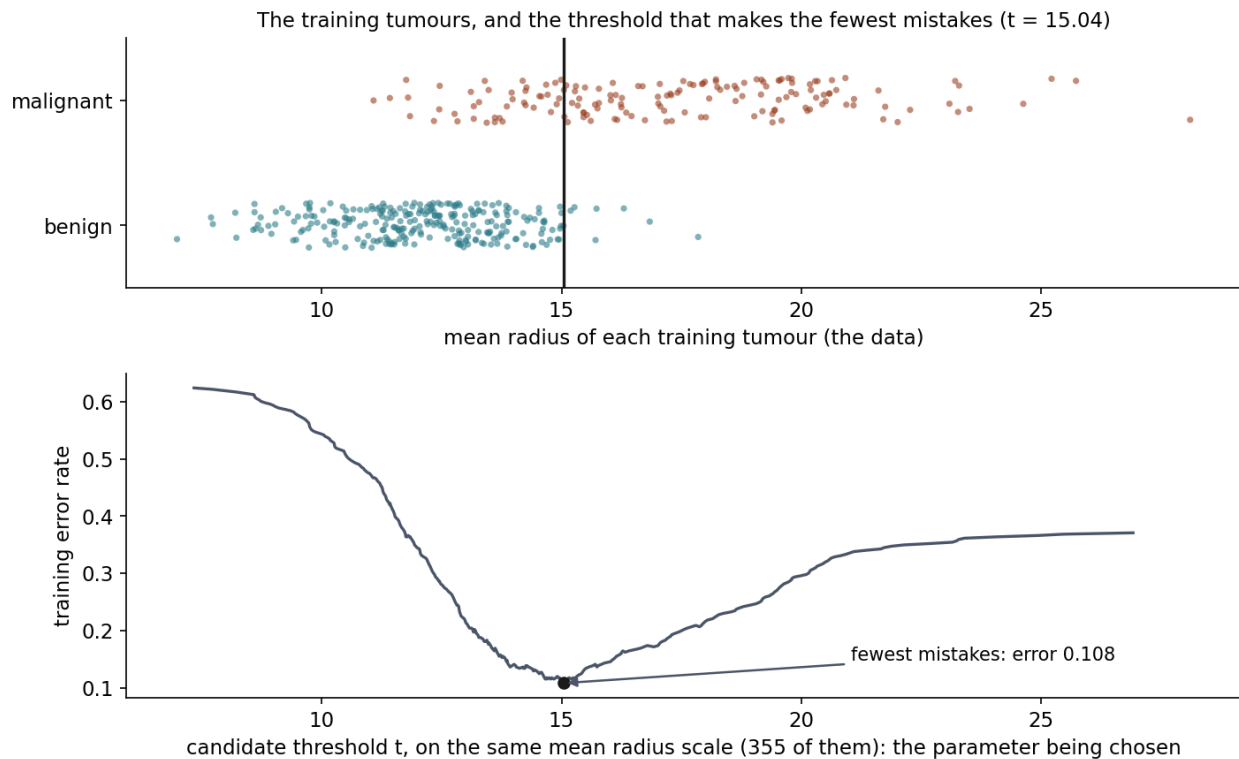
Slide 19 — Learning, in one line

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \hat{R}_S(f)$$

The whole of “learning” in one formula, read left to right as a sentence. $\mathcal{F}$ is the **family**: every model we are willing to consider, for example all the threshold rules of the next slide, or all the weighted sums of the 30 features in notebook 02. $\hat{R}_S$ is the empirical risk, the training error. $\arg \min$ means **the member of the family where that error is smallest**: the model, not the value of the error. $\hat{f}$, with the hat, is that model, an estimate picked from data of the f we really want.

So two decisions are made before any data arrives, which family and which loss, and the data makes the third: which member of the family wins.

Slide 20 — Learning is a search



Top, the training tumours along one feature, malignant above and benign below, and the threshold the search kept. Bottom, the training error of every candidate threshold: learning is walking along this curve and keeping its lowest point. The two panels share their horizontal axis because the parameter is itself a radius value: on top the axis holds each tumour's radius, the data; at the bottom it holds the candidate threshold t , the parameter being chosen.

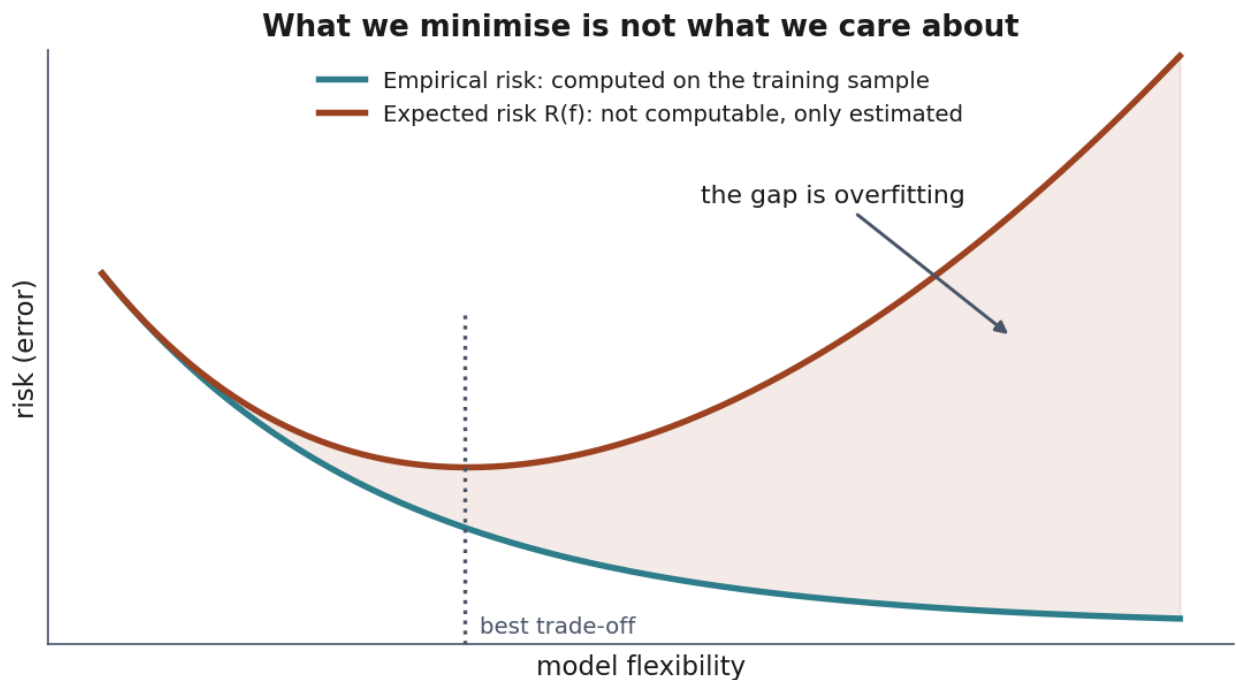
The $\arg \min$ of the previous slide, taken one candidate at a time, on the smallest model there is. One feature, the mean radius of the tumour; one family of rules, "malignant if the radius is above a threshold t ". Every value of t is a different model.

Read the two axes first, because they share one scale and are not the same thing. In the top panel the axis holds **each training tumour's radius**: the data, 426 tumours, malignant above and benign below. In the bottom panel the same axis holds **the candidate threshold t** : the parameter being chosen. They can share an axis only because this parameter is itself a value of the radius. The curve is the training error of each of the 355 candidates, one threshold halfway between each pair of neighbouring values. Learning, here, is literally this: try them all and keep the one with the fewest mistakes. It is $t = 15.04$, with a training error of 0.108, so a training accuracy of 0.892; on the 143 test tumours it never saw, it scores 0.888.

The same rule as a linear model. "Malignant if radius $> t$ " is "malignant if $w_1 \cdot \text{radius} + b > 0$ " with $w_1 = 1$ and $b = -t$. The coefficient is fixed, so the one number being searched for is the intercept: $b = -15.04$. Fixing $w_1 = 1$ loses nothing: any positive w_1 gives the same rule ($w_1 = 2$ with $b = -30.08$ is still "radius above 15.04"), so with a single feature only the ratio $-b/w_1$, the threshold, matters. In lesson 2 the parameters become two coefficients, and this curve becomes a surface.

The real model of notebook 02 does the same thing, choose a family, score every member on the training data, keep the best, with all 30 features, a far richer family and a smarter search than trying every candidate (lesson 3), and reaches 0.986. Here training and test accuracy are close, 0.892 and 0.888, because a one-threshold family is too small to memorise anything. With a richer family that stops being true, which is the next slide.

Slide 21 — The gap that explains everything



The quantity we want and the quantity we can compute are not the same. Read the legend carefully: the curves are two different quantities, not one quantity measured on two datasets. The blue one is an average over the sample in hand; the red one is an average over the whole distribution, which no dataset gives you. Everything in this course is about keeping the gap between them small and honest.

This is probably **the theoretically central slide of the lesson**.

The green curve is: $\hat{R}_S(f)$

that is, the empirical risk, computed on the data we are working with.

The brown curve represents: $R(f)$,

the expected risk.

As the model's flexibility increases, the training error tends to decrease.

If the family of models is flexible enough, it can even reach: $\hat{R}_S(f) = 0$.

But this does not imply: $R(f) = 0$.

On the contrary, beyond a certain point the true risk can increase.

That divergence is what we call **overfitting**.

An important mathematical precision The statement in the notes that the empirical risk falls *monotonically* with flexibility must be read under an implicit condition: we are imagining **nested hypothesis classes** and ideal optimisation.

If:

$$\mathcal{F}_1 \subseteq \mathcal{F}_2 \subseteq \mathcal{F}_3,$$

then certainly:

$$\min_{f \in \mathcal{F}_3} \hat{R}(f) \leq \min_{f \in \mathcal{F}_2} \hat{R}(f) \leq \min_{f \in \mathcal{F}_1} \hat{R}(f).$$

Because a larger family can always choose the solution of the smaller family as well.

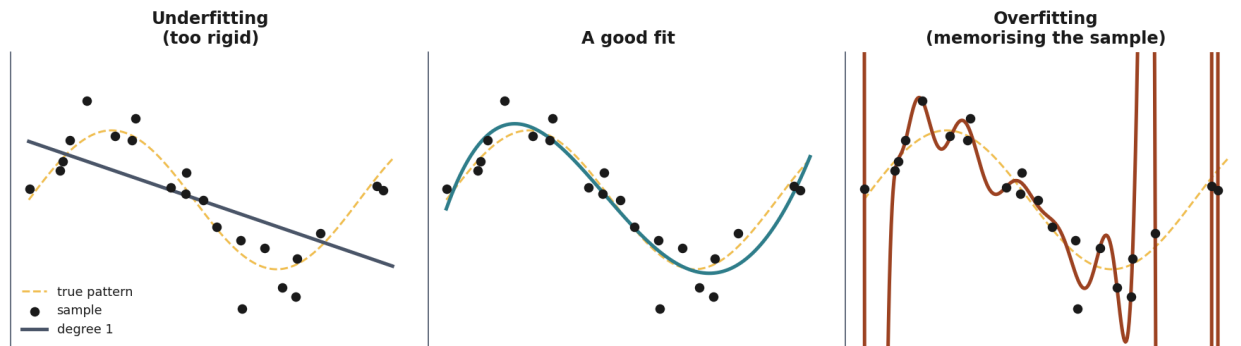
In practice, with imperfect optimisation algorithms or non-nested families, perfect monotonicity is not guaranteed.

The handout (Section 2.2) shows both halves on the twenty-two points of the overfitting figure. The best training mean squared error falls from 0.259 at degree 1 to 0.054 at degree 4 and 0.014 at degree 18, as nesting promises. But `np.polyfit` at degree 21, twenty-two coefficients for twenty-two points, enough to pass through every one of them, returns a training error of 0.0152, *higher* than its own 0.0149 at degree 20: the powers x^0 to x^{21} are so nearly parallel that the solver loses the exact answer to rounding. Written in a better-conditioned basis, degree 21 reaches zero. The fall is a property of the families; what a solver delivers can fall short of it.

And the brown curve, if $R(f)$ cannot be computed? The answer in the notes is correct and important: it is **conceptual**.

In a simulated experiment, where we know the generating distribution, we can actually estimate the true risk almost arbitrarily well. In a real problem we cannot.

Slide 22 — The lowest training error is the worst model



The same twenty-two points fitted three times. The dashed line is the pattern the data really came from, which no method gets to see. Left, a straight line is too rigid to follow it. Right, a degree-18 polynomial passes almost exactly through every sample point and shoots off the scale between them — its empirical risk is nearly zero and it has learnt the noise, not the pattern. The middle one is what we want, and nothing in the training error distinguishes it from the right-hand one.

Three cases:

underfitting → **good fit** → **overfitting**

In the first case the model is too rigid.

For example a straight line: $f(x) = ax + b$

trying to approximate a clearly curved relationship.

This produces **high bias**.

In the third case a very high-degree polynomial passes almost perfectly through the samples:

$$f(x) = \sum_{k=0}^{18} a_k x^k.$$

The training error becomes very low, but between the samples the function oscillates violently.

So:

lowest training error $\neq$ best generalisation

It is an excellent way of breaking, right away, the typical programmer's intuition:

“if the model reproduces the data perfectly, then it works well”.

No: it may have learned the data instead of the phenomenon.

Slide 23 — Which gives us the one rule for today

Hold out part of the data. Never let the fitting procedure touch it. Measure there.

This is the operational solution to the previous problem.

We split: $S = S_{\text{train}} \cup S_{\text{test}}$

with: $S_{\text{train}} \cap S_{\text{test}} = \emptyset$.

The model is built using only: S_{train} .

Then we measure:

$$\hat{R}_{\text{test}}(f) = \frac{1}{m_{\text{test}}} \sum_{i \in \text{test}} L(f(x_i), y_i).$$

If the test set did not take part in choosing f , then:

$$\mathbb{E}[\hat{R}_{\text{test}}(f) \mid f] = R(f).$$

This is the mathematical reason why the test set works.

Careful with the meaning of “unbiased” It does not mean that our particular value is equal to $R(f)$.

It means that over many possible independent test sets: $\mathbb{E}[\hat{R}_{\text{test}}] = R(f)$.

So we might obtain: 0.96, 0.98, 0.95, 0.99, ...

but the distribution of the estimates is centred on the correct value.

Standard error of the accuracy For a proportion $\hat{p}$, approximately:

$$SE(\hat{p}) = \sqrt{\frac{\hat{p}(1 - \hat{p})}{m}}.$$

With: $\hat{p} = 0.986$, $m = 143$

we get:

$$SE \approx \sqrt{\frac{0.986 \cdot 0.014}{143}} \approx 0.0098.$$

So almost **one percentage point**.

Saying:

`accuracy = 0.986`

therefore does not mean really knowing the accuracy to the third decimal place.

The note that with only 2 errors the normal approximation is unreliable is also very correct; a binomial/Wilson interval is more appropriate.

Question — who is p ?

The notes of this slide say “for a proportion $\hat{p}$, approximately” and give the standard error. Who is p , and who is $\hat{p}$?

Here $\hat{p}$ is simply **the proportion observed in the sample**. In our case, it is the **accuracy measured on the test set**.

So, if the test set has 143 examples and 141 are classified correctly: $\hat{p} = \frac{141}{143} \approx 0.986$.

The symbol with the “hat”, $\hat{p}$, indicates that we do not know the true probability p , but we are **estimating it from the data**.

The distinction is:

p = true probability of classifying a new example correctly

whereas

$\hat{p}$ = accuracy observed on the test set.

In the context of this slide, ideally p is tied to the model’s true behaviour on the distribution $\mathcal{D}$, so: $p = P(f(X) = Y)$.

If we use the zero-one loss, then the true risk is: $R(f) = P(f(X) \neq Y)$,

so that: $p = 1 - R(f)$.

But we do not know p , because we do not know $\mathcal{D}$. What we observe is only $\hat{p}$.

The formula in the notes:

$$SE(\hat{p}) \approx \sqrt{\frac{\hat{p}(1 - \hat{p})}{m}}$$

estimates how much **the observed accuracy $\hat{p}$ would fluctuate from one test set to another**.

The theoretical formula would actually be:

$$SE(\hat{p}) = \sqrt{\frac{p(1 - p)}{m}},$$

but since p is unknown, we replace it with its estimate $\hat{p}$: $p \approx \hat{p}$.

In our example: $\hat{p} = 0.986$, $m = 143$,

so:

$$SE(\hat{p}) \approx \sqrt{\frac{0.986(1 - 0.986)}{143}} \approx 0.0098.$$

That is, about **0.98 percentage points**.

So the meaning is: if we repeated the experiment many times with new independent test sets of 143 patients, the measured accuracy would typically change by roughly one percentage point around the true accuracy p .

In short:

$$p = \text{true, unknown accuracy}, \quad \hat{p} = \text{measured accuracy}$$

and in statistics the hat $\hat{}$ means precisely **“estimate of an unknown quantity”**.

Slide 24 — When *not* to use machine learning

This is an important slide because it avoids turning ML into a universal solution.

If the rule is known, let us write it.

For example:

```
is_even = n % 2 == 0
```

There is no need to train a classifier.

Second point: if the dataset does not represent the deployment situation, there is no reason for the model to work.

Third: if an error is at the same time:

**certain to happen sooner or later,
irreversible,
not supervised by a person,**

the architectural problem comes before the accuracy.

A precision of 99.999% does not remove the fact that: $P(\text{error}) > 0$.

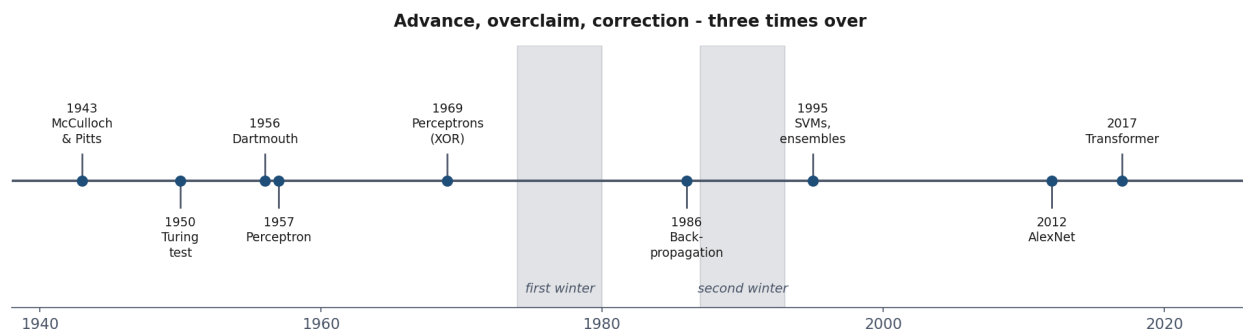
Fourth point: the dataset might embody systematically unjust decisions.

Fifth: a simple baseline might already solve the problem.

So the slide invites us to apply the principle:

before asking *which model*, ask *whether a model is needed at all*.

Slide 25 — Seventy years in one picture



Seventy years in one picture. The pattern worth recognising is the rhythm: a genuine advance, claims well beyond the evidence, and a correction expensive enough to cost a generation of funding.

The historical section begins.

The timeline runs from McCulloch–Pitts to the Transformer.

The two grey bands are the **AI winters**.

But history is not presented as a list of dates: the pattern to bring out is:

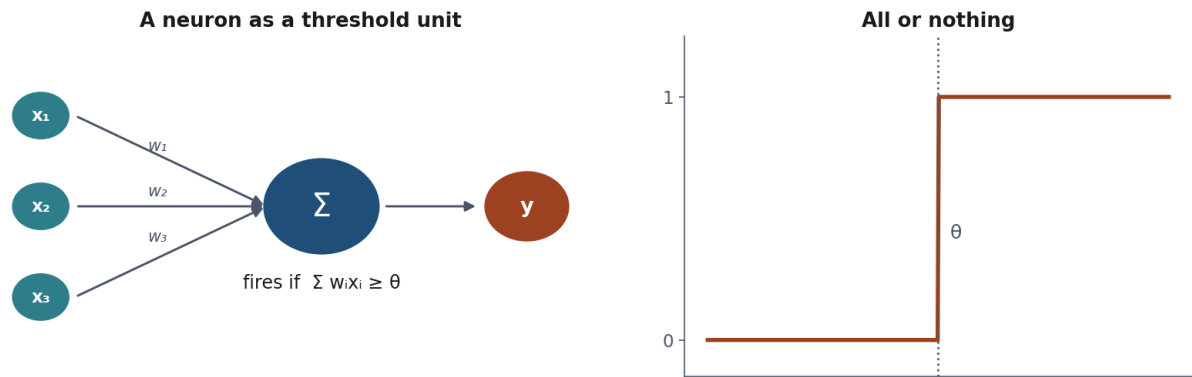
advance → overclaim → correction.

It is perfectly connected to the course’s methodology.

The historical problem being highlighted is not simply “the technology did not work”.

It often worked, but the conclusions drawn were far broader than what the experiment showed.

Slide 26 — 1943: the first artificial neuron



1943: the first artificial neuron — inputs, weights, a threshold.

The McCulloch-Pitts neuron computes:

$$z = \sum_i w_i x_i$$

and produces:

$$y = \begin{cases} 1 & z \geq \theta \\ 0 & z < \theta. \end{cases}$$

Equivalently we can write: $y = H(w^T x - \theta)$,

where H is a step function.

The important historical step is that this unit implements a logical decision.

But the weights are not yet **learned**.

So we have a computational neuron, not yet machine learning in the modern sense.

The notes connect this object very well to the later lessons.

The perceptron will use the same scheme.

Logistic regression will replace the step function with a sigmoid:

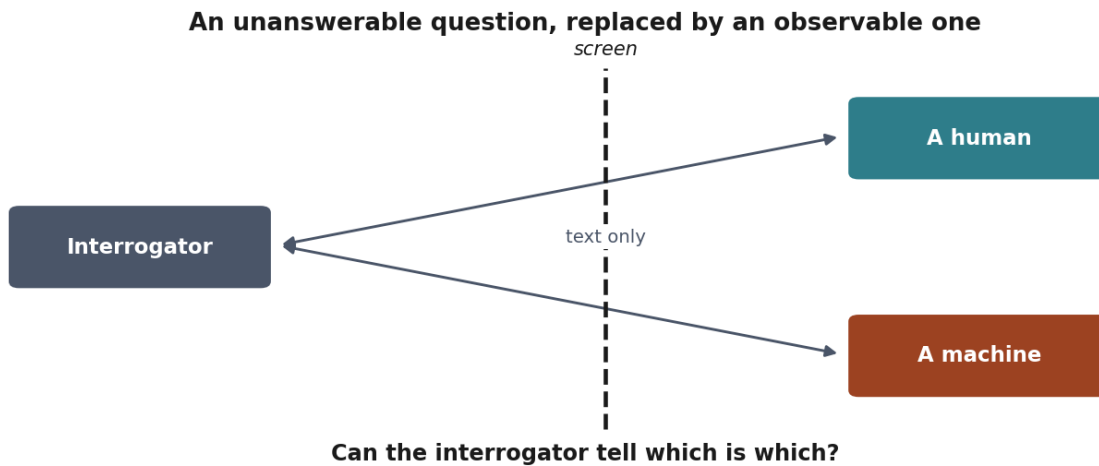
$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

A modern network will often use ReLU: $\text{ReLU}(z) = \max(0, z)$.

The underlying structure remains:

$$x \rightarrow w^T x + b \rightarrow \text{nonlinearity}.$$

Slide 27 — 1950: Turing’s imitation game



1950: Turing’s imitation game, which replaced “can machines think” with a question that can actually be settled.

The real lesson of the slide is not simply “what the Turing Test is”.

The point is **operationalisation**.

The question:

Can machines think?

is hard even to define.

Turing replaces it with something observable.

This idea is connected to ML metrics.

Questions such as:

“is this model good?”

are also too vague.

We replace them with:

accuracy, recall, MSE, AUC, ...

But a metric is a **proxy**.

The epistemological lesson is:

what I measure $\neq$ necessarily what I care about

even when the measurement is perfectly defined.

Slide 28 — 1956: Dartmouth

The seven topics of the Dartmouth proposal

Automatic computers	
Language use	
Neuron nets	"a 2 month, 10 man study"
Theory of computation	
Self-improvement	Seventy years later, most of this list is still open.
Abstractions	
Randomness and creativity	

1956: Dartmouth, where the field acquired both its name and its habit of optimistic timelines.

Here the term *Artificial Intelligence* is formally born.

The part that matters is the project's enormous optimism:

"a 2 month, 10 man study"

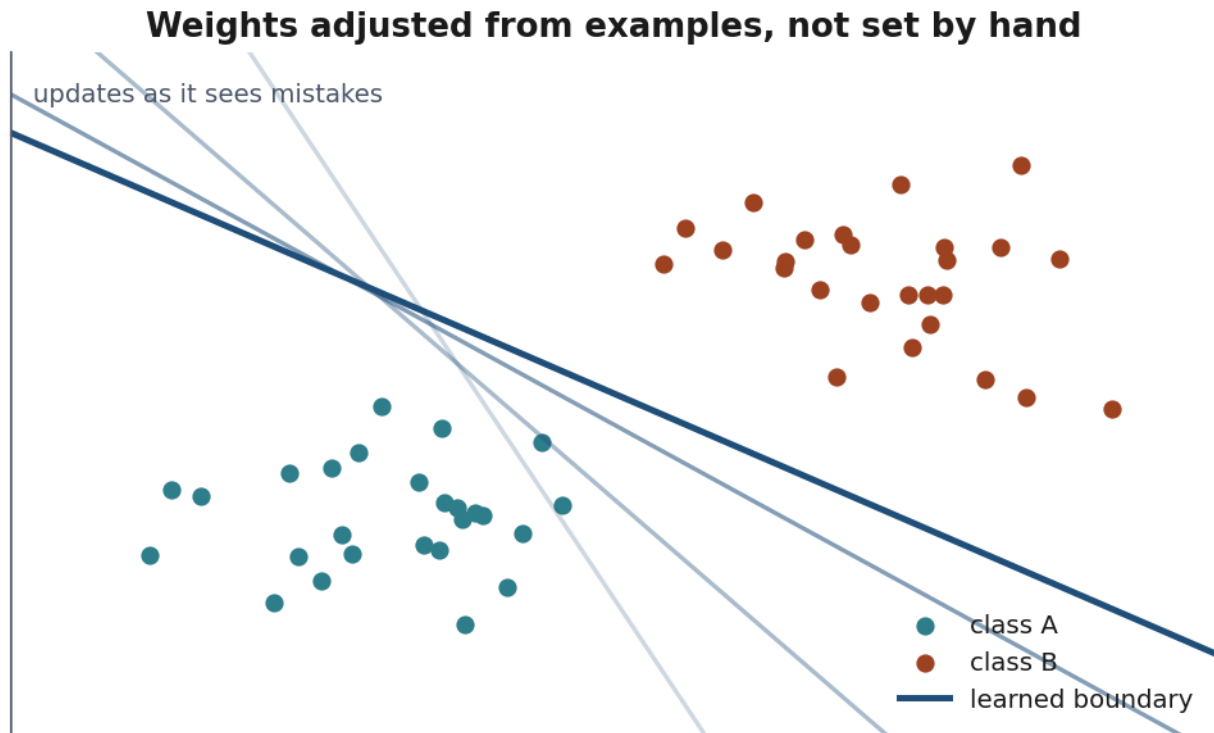
for topics that include language, neurons, self-improvement, abstraction and creativity.

The slide does not want to ridicule the researchers; it wants to show how easily an initial discovery can be extrapolated.

It is the first historical example of the theme:

measured result vs extrapolated promise.

Slide 29 — 1957: the perceptron learns



1957: the perceptron — the first machine that improved with experience rather than with rewriting.

Now the learning of weights finally appears.

In the perceptron we can write:

$$\hat{y} = \text{sign}(w^T x + b).$$

When an example is misclassified:

$$w \leftarrow w + \eta y x.$$

Depending on the convention used for the labels, the formula can take slightly different forms, but the principle is always:

an error changes the weights in the direction that makes it more likely to classify that example correctly next time.

The figure shows exactly how the boundary evolves.

The essential part of the notes is the condition:

if linearly separable

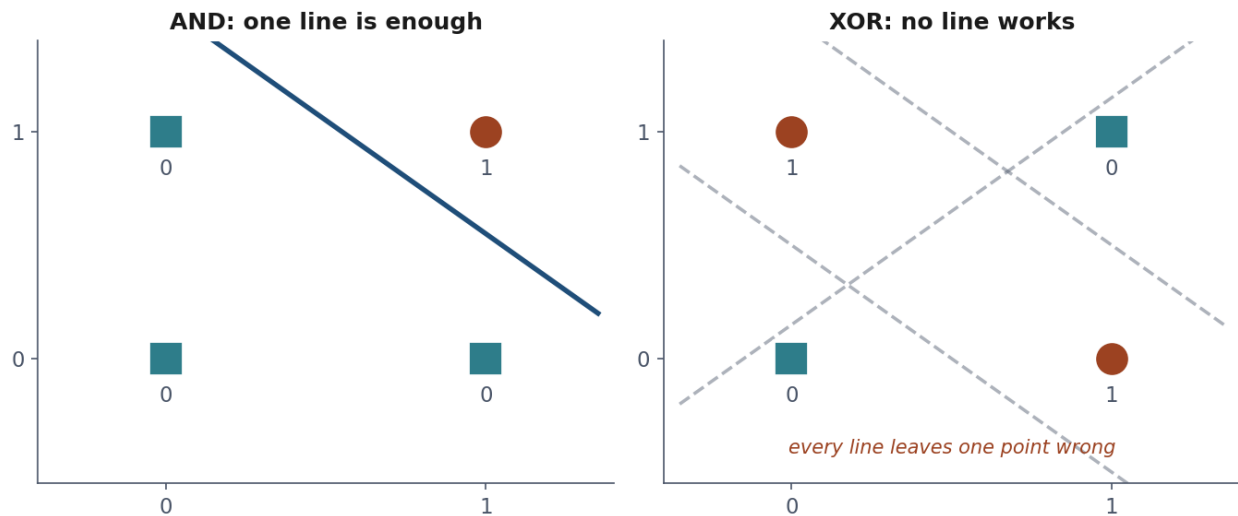
The perceptron convergence theorem does not say that the perceptron solves every classification problem.

It essentially says:

linear separability $\Rightarrow$ convergence in a finite number of updates.

That condition will become crucial on the next slide.

Slide 30 — 1969: the first winter



1969: *XOR*, a function a single-layer perceptron cannot represent, and the book that ended the first wave of funding.

The XOR problem is:

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Geometrically, the points of the two classes occupy alternate corners of a square.

So there is no line: $w_1x_1 + w_2x_2 + b = 0$

that separates them.

So a single perceptron cannot represent XOR.

But this does not mean a neural network cannot It is a crucially important distinction.

With a hidden layer we can build intermediate functions and obtain XOR. Two hidden units are enough: one computes OR, $h_1 = [x_1 + x_2 \geq 0.5]$, another computes AND, $h_2 = [x_1 + x_2 \geq 1.5]$, and the output fires when the first is on and the second off, $y = [h_1 - h_2 \geq 0.5]$, where $[\cdot]$ is 1 when the condition holds and 0 otherwise. The four inputs give 0, 1, 1, 0. Geometrically, no single line separates the two classes, but the strip between the two parallel lines $x_1 + x_2 = 0.5$ and $x_1 + x_2 = 1.5$ contains exactly the points where XOR is 1. What was missing in 1969 was not a network that could do it but a way to learn such weights.

So the limitation was:

single-layer linear threshold network

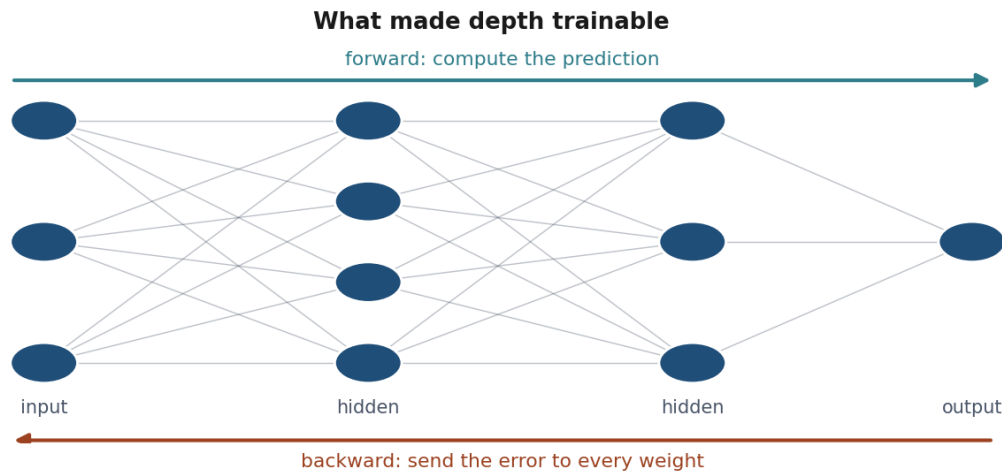
not:

all neural networks.

The notes point precisely to the historical problem: a correct result on a restricted case was read far more broadly.

And it is exactly the same fallacy to avoid when interpreting a benchmark.

Slide 31 — 1986: backpropagation



1986: backpropagation — the chain rule applied to a network, which made several layers trainable at last.

Backpropagation makes training multilayer networks practical.

Forward pass:

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow \hat{y}.$$

Then we compute: $L(\hat{y}, y)$.

Backward pass: $\frac{\partial L}{\partial w}$

for every weight in the network.

The key idea is the **chain rule**.

If:

$$L = L(a), \quad a = a(z), \quad z = z(w),$$

then:

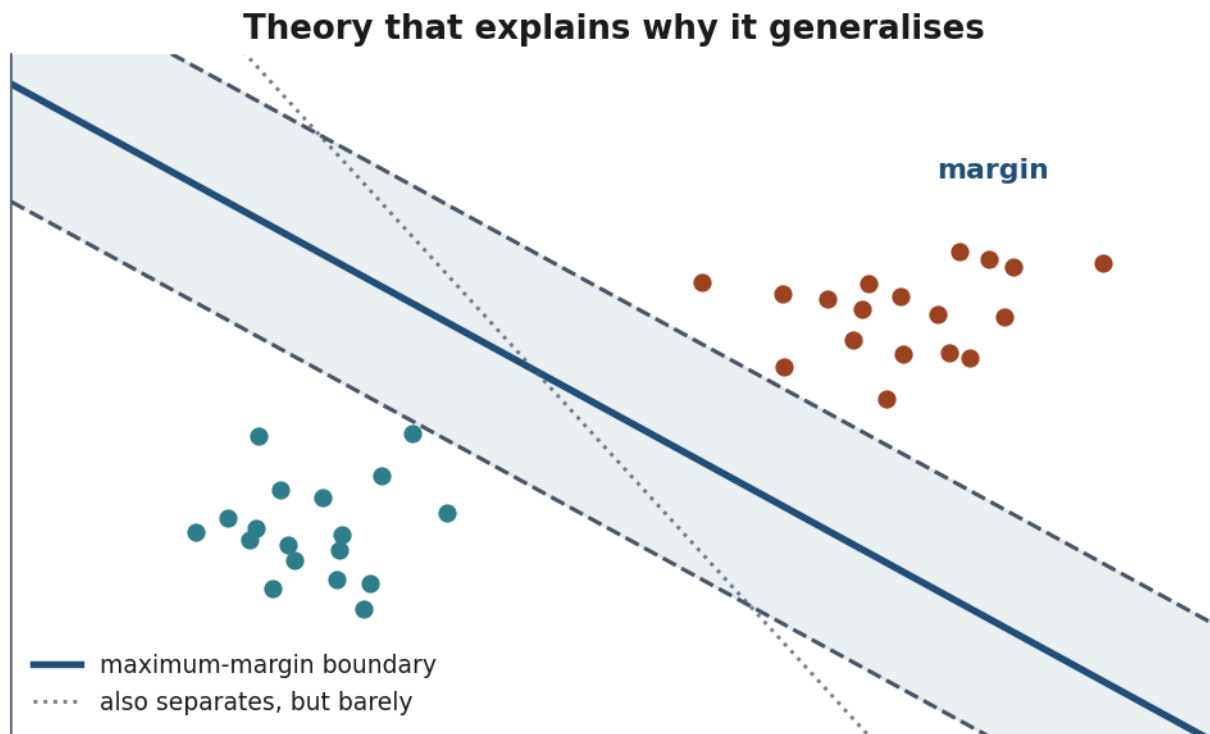
$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w}.$$

This makes it possible to assign “responsibility” for the error even to hidden units that do not see the target directly.

So the sentence in the notes is correct:

the 1969 problem was not only expressiveness; above all it was finding an efficient way to train the hidden units.

Slide 32 — 1995-2010: statistics takes over



1995 onwards: support vector machines and the statistical turn, where theory caught up with practice for about fifteen years.

Here the paradigm changes.

From:

simulating intelligence

to:

estimating functions from data.

It is the historical phase to which much of the course belongs: SVMs, ensembles, bias-variance, cross-validation.

The figure introduces the **maximum margin** intuitively.

Among many lines that separate the classes perfectly, the SVM looks for the one that maximises the distance from the nearest points.

Writing the hyperplane as $w^T x + b = 0$, the geometric margin is proportional to: $\frac{1}{\|w\|}$.

The hard-margin SVM can be formulated as: $\min_{w,b} \frac{1}{2} \|w\|^2$

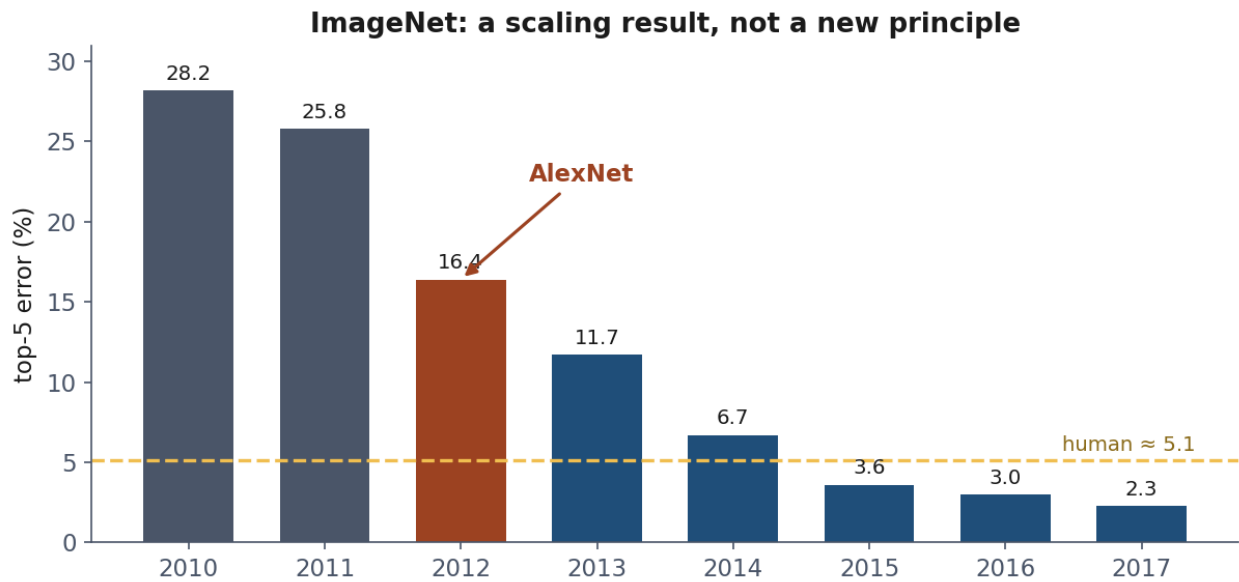
subject to: $y_i(w^T x_i + b) \geq 1$.

The intuitive part is:

two separators have zero training error, but they are not necessarily equally convincing about the future.

This connects the SVM perfectly to the theme of generalisation.

Slide 33 — 2012: AlexNet



2012: AlexNet. Conceptually little was new — what had changed was data and compute.

The slide's thesis is:

AlexNet was above all a result of **scaling**, not the invention of the CNN from scratch.

Convolutional networks already existed.

What changes drastically is the combination of:

ImageNet → a lot of data,

GPUs → a lot of computing power,

ReLU, dropout, augmentation → important algorithmic details.

The chart shows the top-5 error falling:

2010: 28.2%

2011: 25.8%

2012 AlexNet: 16.4%

2013: 11.7%

2014: 6.7%

2015: 3.6%

So the point is not only the 2012 jump.

After that result **the whole field changes method**, and improvement continues rapidly.

An excellent historical link to the difference between: new principle

and

same principle + data + compute.

Slide 34 — The pattern

The table sums up:

Advance	Overclaim	Correction
perceptron learns	machines that walk and talk	XOR / winter
backprop trains depth	networks solve everything	no data / second winter
scale delivers	?	?

The last two cells are deliberately left open.

The message is not to predict another AI winter.

It is to distinguish:

evidence

from:

extrapolation.

The same question we must ask of an article about AI:

what was actually measured?

must also be asked of our own notebooks.

Slide 35 — Three kinds of learning

Here the taxonomy appears:

supervised,
unsupervised,
self-supervised.

The important sentence is:

the taxonomy depends on **where the target comes from**, not necessarily on the algorithm.

It is a very useful clarification.

For example, a regression can be used:

- supervised, if the target is an external label;
- self-supervised, if the target is a part of the input we have hidden.

So the nature of learning is not defined simply by `LinearRegression` vs `KMeans`.

Slide 36 — Supervised

Every example contains: (x_i, y_i) .

The target y_i comes from outside.

If: $y \in \{1, \dots, K\}$

we have classification.

If: $y \in \mathbb{R}$

we have regression.

The great advantage is that we can compare: $\hat{y}$

with: y .

So evaluation is relatively well defined.

The great cost is obtaining y .

Labels require people, time, experts or measurements.

ImageNet is an excellent example: the existence of large quantities of images was not enough.
Annotated images were needed.

Slide 37 — Unsupervised

Here we have no target: $X = \{x_1, \dots, x_m\}$.

We look for structure.

Clustering: $x_i \rightarrow$ cluster.

Dimensionality reduction:

$$\mathbb{R}^n \rightarrow \mathbb{R}^k, \quad k < n.$$

Anomaly detection: $x_i \rightarrow$ degree of abnormality.

A very important point from the notes It is not true that unsupervised learning has no objective function.

K-means, for example, minimises:

$$J = \sum_{i=1}^m \|x_i - \mu_{c_i}\|^2.$$

The problem is that a low value of J does not prove that the clusters mean anything in the domain.

We can even obtain: $J = 0$

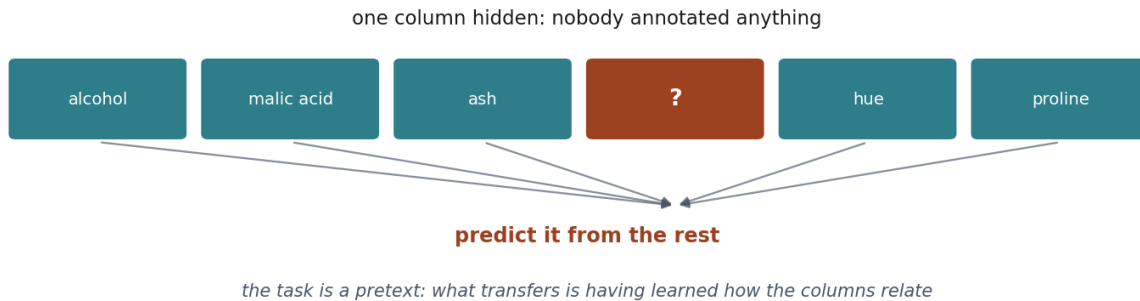
by putting every point in its own cluster.

So the same theme returns:

optimising a metric $\neq$ solving the real problem.

Slide 38 — Self-supervised

Supervision that costs nothing



Where the labels come from when nobody labelled anything: hide part of the input and ask the model to reconstruct it.

Here the target is built from the input.

Imagine: $x = (x_1, x_2, x_3, x_4, x_5)$.

We hide x_4 .

We use: (x_1, x_2, x_3, x_5)

to predict: $y = x_4$.

So formally we are back to supervised learning: $\tilde{x} \rightarrow y$.

The difference is that nobody annotated the target.

It is already present in the data.

Pretext task Predicting a hidden feature is not necessarily the real goal.

Its function is to force the model to learn a useful representation.

In language:

context $\rightarrow$ hidden token.

In images:

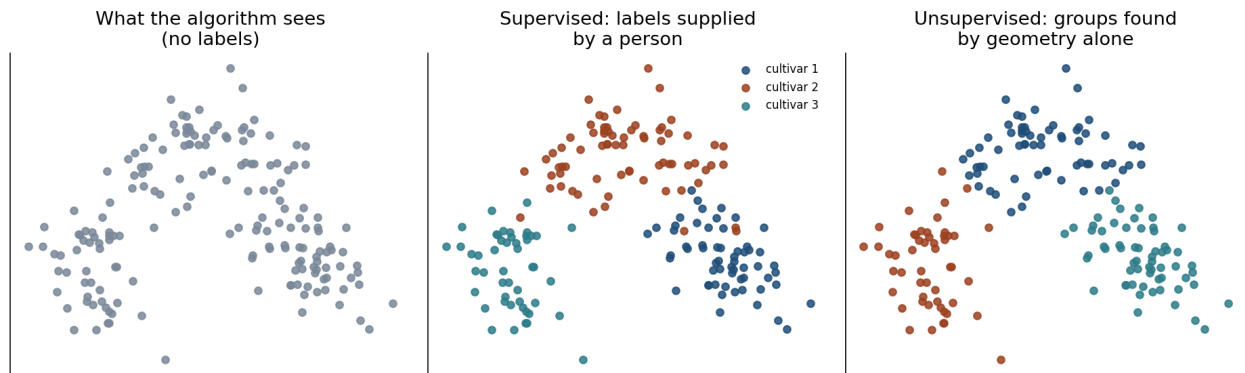
visible patches $\rightarrow$ hidden patch.

The big economic consequence is that:

labels effectively free

makes it possible to exploit enormous quantities of raw data.

Slide 39 — One dataset, three questions



One dataset, three questions. What changes is not the data but what you ask of it.

The same 178 wines are looked at in three ways.

In the first plot the model simply sees points.

In the second, human labels appear.

In the third, k-means finds geometric clusters.

An important clarification: k-means cluster IDs are arbitrary.

Cluster 0 is not cultivar 1 in any intrinsic sense.

A permutation:

$$0 \rightarrow 2, \quad 1 \rightarrow 0, \quad 2 \rightarrow 1$$

would represent exactly the same solution.

This is why metrics such as the **Adjusted Rand Index** compare partitions without requiring the label numbers to coincide.

An ARI of 0.897 is very high, but the notes rightly warn that this is a particularly favourable dataset.

Slide 40 — Where the target comes from

A nice summary table.

Supervised: target from person

Self-supervised: target from input

Unsupervised: no target.

The really interesting column is **Measurable?**

In the supervised case we can check directly whether we guessed the target.

In the unsupervised case there is not necessarily a “true answer”.

In the self-supervised case we can evaluate the pretext task very well, but this does not necessarily mean the representation is useful for the downstream task.

Example: excellent masked-token accuracy

does not automatically imply: excellent usefulness on every later task.

Slide 41 — Notebook 1, live

The slide's body is a **toolbox table**: each of the three questions asked of the same 178 wines, the call that answers it and the number it reaches.

- **Supervised**, predict the cultivar: a pipeline of `StandardScaler()` and `LogisticRegression()`, scored with `accuracy_score`, **0.981**;
- **unsupervised**, labels thrown away: `KMeans(n_clusters=3)`, drawn with PCA, scored against the hidden labels with the adjusted Rand index, **0.897**;
- **self-supervised**, hide `flavanoids` and predict it from the other measurements: `LinearRegression`, scored with `r2_score`, $R^2 = \mathbf{0.816}$ (1 is perfect, 0 is no better than always guessing the mean).

The point is not to compare these values numerically: they measure different things.

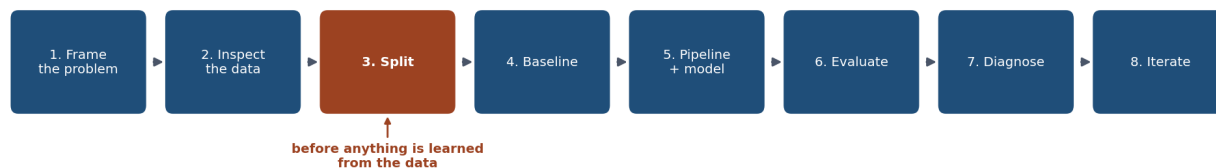
The point is that the dataset is practically identical.

What changes is the definition of the problem.

It is an excellent demonstration of the sentence on slide 35:

the kind of learning is determined above all by the relationship between input and target.

Slide 42 — The workflow



The cycle, and the order that matters most: the split comes before anything is fitted, not after.

The workflow is:

Frame → **Inspect** → **Split** → **Baseline** → **Pipeline/model** → **Evaluate** → **Diagnose** → **Iterate**

The slide strongly highlights point 3:

split before anything is learned from the data

This means the workflow is not simply an organisational diagram.

Some operations **do not commute**.

In general:

$$\text{split}(\text{fitTransform}(X))$$

is not methodologically equivalent to:

$$\text{fitTransform}(X_{\text{train}})$$

followed by transforming the test set.

This will become the theme of leakage.

Slide 43 — Notebook 2, live

The slide's body is a **toolbox table**: each step of the workflow beside the call that performs it in the notebook and the slide that introduces the idea, from `load_breast_cancer` and `train_test_split(X, y, test_size=0.25, stratify=y)` to `DummyClassifier`, `make_pipeline` with `StandardScaler()` and `LogisticRegression()`, `.score`, `ConfusionMatrixDisplay`, `classification_report` and `predict_proba`. If you get lost in a cell, the table leads back to the slide.

The notebook runs **alongside** the next twelve slides rather than instead of them: each step is a slide and then the cell that performs it. Three moments to stop on. **The split**, because it happens before anything is learned, which is the whole rule of the day. **The baseline at 0.629**, because it is what makes 0.986 mean something. And **the threshold sweep** at the end, where the same model, with the same weights, goes from missing no malignancy with eleven false alarms to missing seven with none.

Slide 44 — Step 1: frame it

First question:

what must we predict, using what information?

Second:

who will use the prediction, and what decision must they take?

Third:

which error costs more?

These are questions that come before any algorithm.

In the example:

measurements $\rightarrow$ malignant/benign.

But the system is presented as a **screening aid**, not a substitute for diagnosis.

This completely changes the cost structure.

If the model serves for screening, it may be reasonable to tolerate more false positives in order to reduce false negatives.

So the metric should follow from the problem, not be chosen after seeing the results.

Slide 45 — Which error is worse?

Accuracy weights these equally. Nobody else does.

	predicted benign	predicted malignant
benign	correct no action needed	false alarm anxiety, a follow-up test
malignant	missed case a patient sent home who should not be	correct treated in time

Two of these four cells are failures, and they are not the same size: a false alarm costs a follow-up test, a missed case sends a patient home who should not go. Accuracy adds all four up as though they weighed the same, which is why the metric cannot be chosen until this question has been answered.

Here the confusion matrix appears, conceptually.

If we define “malignant” as the positive class: TP = malignant correctly identified

FN = malignant predicted benign

FP = benign predicted malignant

TN = benign correctly identified.

Accuracy is: $\frac{TP+TN}{TP+TN+FP+FN}$.

This formula implicitly gives the same weight to: FP

and

FN .

But in the application they do not necessarily have the same cost.

We can think of a more general loss:

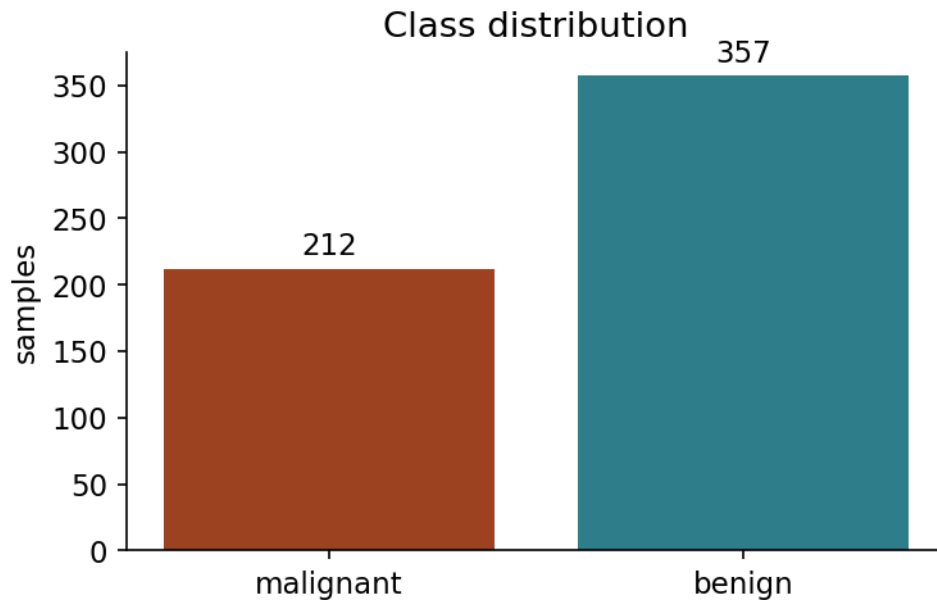
$$L = c_{FN}FN + c_{FP}FP$$

with, for example: $c_{FN} \gg c_{FP}$.

So mathematics does not decide the cost.

It represents it after the domain has decided it.

Slide 46 — Step 2: look first



Look before you touch anything. The class balance decides which metrics will mean something later: 357 benign against 212 malignant, so answering “benign” every time is already right 62.7% of the time.

The dataset contains: 357 benign

and

212 malignant.

Total: 569.

If we always predict benign:

$$accuracy = \frac{357}{569} \approx 0.6274.$$

So:

62.7%

is the trivial baseline.

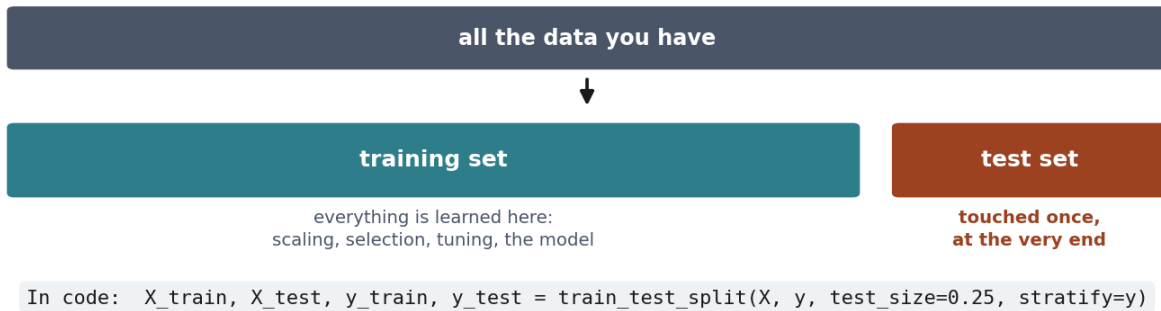
It matters because a result like 70% might look decent in absolute terms, but against 62.7% it is far less impressive.

So this slide introduces the idea:

absolute score vs improvement over a baseline.

Slide 47 — Step 3: split, before anything else

Split first, then never look right again until you are finished



Train and test today; lesson 5 adds the validation set that every choice should actually be made on.

The key sentence:

Before scaling. Before selecting features. Before looking at correlations with the target.

Why?

Because all these operations can **learn something from the data**.

For example the StandardScaler estimates:

$$\mu_j = \frac{1}{m} \sum_i x_{ij}$$

and

$$\sigma_j = \sqrt{\frac{1}{m} \sum_i (x_{ij} - \mu_j)^2}.$$

If we also use the test samples to compute μ_j and σ_j , some information about the test set enters the transformation.

It is a weak leak compared with reading the labels directly, but the principle is identical.

Stratification With `stratify=y`, the ratio between classes is kept approximately constant:

$$P_{\text{train}}(y) \approx P_{\text{test}}(y) \approx P_{\text{full}}(y).$$

It is particularly important with small or very imbalanced datasets.

Question — what is a single test score worth?

The notes of the split slide say the test set is ‘touched once, at the very end’...the accuracy computed on the test set depends on that particular sample of the test set. so changing the sample would change that value too. so what value should I attach to a score computed on a single test set? how should a single score be read?

Exactly: **a score computed on a single test set is not “the true performance of the model”**. It is an **estimate** of the true performance, obtained from one particular finite sample. And this is exactly where it pays to be very precise with the students.

You can put it like this:

The test score is an honest measurement, but not an exact one. It is honest because the test set did not influence the model; it is not exact because it depends on the particular sample we happened to draw.

This is the fundamental distinction.

If we write

$$p = P(f(X) = Y)$$

for the model’s **true accuracy** on the future distribution, p is unknown.

On the test set, instead, we observe:

$$\hat{p} = \frac{\text{number of correct predictions}}{m_{\text{test}}}.$$

So $\hat{p}$ is only an estimate of p .

If we repeated the experiment with many independent test sets of the same size, we would get different values: $\hat{p}_1, \hat{p}_2, \hat{p}_3, \dots$

some a little above p , others a little below.

The advantage of the independent test set is that, on average,

$$\mathbb{E}[\hat{p}] = p.$$

This is the idea of an **unbiased estimate**.

So we must not interpret: $\hat{p} = 0.986$

as:

“the true accuracy is 98.6%”.

Better to say:

“On this independent test set we observed an accuracy of 98.6%. It is our estimate of the generalisation performance, but it carries sampling uncertainty.”

This formulation is much more correct.

In the case of the slide, you have 143 test examples. The score is: $\hat{p} = \frac{141}{143} = 0.986$.

But with another test set of 143 patients you might have observed, for example 0.972, 0.993 or 0.965, without changing the model in the slightest.

So the test set solves one problem, but not all problems:

independent test $\Rightarrow$ no selection bias

but it does not imply:

no sampling variability

This is probably the most important sentence to get across.

So what does “touched once, at the very end” mean? It does not mean:

“measure it once because that measurement is perfect”.

It means:

“measure it once because every time you look at the test set and use what you saw to change something, you start selecting the model on the test set”.

Imagine doing:

1. model A $\rightarrow$ test accuracy 0.95
2. I change the model
3. model B $\rightarrow$ test accuracy 0.97
4. I change it again
5. model C $\rightarrow$ test accuracy 0.98

At that point you have implicitly used the test set to choose C.

So C is no longer independent of the test set.

You have turned the test set into a **validation set**.

This is why the correct workflow is:

training data $\rightarrow$ model selection $\rightarrow$ final test once

The choice of model must be made beforehand, using training/validation or cross-validation.

In two sentences A very simple formulation:

“The test set does not give us the truth. It gives us an independent measurement of the truth.”

And then:

“If I took another test set, I would get a slightly different number. Independence makes the number not systematically optimistic; the size of the test set decides how noisy the number is.”

This pair of concepts is very effective:

independence controls bias; sample size controls variance.

More precisely:

- independent test set $\rightarrow$ avoids systematic optimism;
- large test set $\rightarrow$ reduces the uncertainty of the estimate.

For an accuracy, roughly:

$$SE(\hat{p}) \approx \sqrt{\frac{\hat{p}(1 - \hat{p})}{m_{\text{test}}}}.$$

So as m_{test} grows, the uncertainty falls like: $\frac{1}{\sqrt{m_{\text{test}}}}$.

This is the link with the standard error under slide 23.

There is also a link with slide 67 Slide 67 shows:

One split is one measurement

and it is exactly the answer to this question.

The single test score is:

one measurement

not:

the performance

This is why cross-validation and stability across splits come later, in lesson 5.

One distinction is worth anticipating:

cross-validation and the final test set do not have exactly the same role.

Cross-validation serves above all during development, to:

- compare models;
- choose hyperparameters;
- estimate variability;
- understand how much the result depends on the split.

The final test set serves to give a **final independent estimate**, after all these decisions.

So you could schematise it like this:

CV: choose and understand

Test set: judge at the end

And for this slide, a very effective sentence in my view would be:

“98.6% is not a property of the model. It is the result of measuring that model on this particular independent sample.”

And right after:

“What independence buys us is honesty, not certainty.”

This second sentence sums up the point almost perfectly.

Slide 48 — Step 4: baseline first

The baseline is:

predict the majority class.

Result: 62.7%.

This baseline is deliberately stupid.

And that is precisely why it is useful.

If our sophisticated system produces: 63.1%

we have achieved practically nothing.

A baseline model therefore serves to establish: how much value does learning really add?

Baselines can of course be more sophisticated than this one, but the rule is:

compare first against the simplest plausible solution.

Slide 49 — Step 5: pipeline, not two steps

The code:

```
model = make_pipeline(  
    StandardScaler(),  
    LogisticRegression(max_iter=5000),  
)
```

is not just convenience.

It is a methodological guarantee.

The scaler must learn: μ_j, σ_j

on the training set.

Then it transforms the test set using **the same ones**:

$$z_j^{test} = \frac{x_j^{test} - \mu_j^{train}}{\sigma_j^{train}}.$$

It must not be:

$$\frac{x_j^{test} - \mu_j^{all}}{\sigma_j^{all}}.$$

The pipeline is particularly important inside cross-validation, because for each fold the statistics are recomputed exclusively on the training folds.

So:

the pipeline makes correct **by construction** something that would be easy to forget by hand.

Slide 50 — What the pipeline prevents

Preprocessing outside the split

the mean was computed over the test rows too



optimistic score, no error, nothing to notice

Preprocessing inside the pipeline

the mean is learned from training rows only



and it is refitted inside every cross-validation fold

The same two steps in the two possible orders. Look at where “split” sits on the top row: after the scaling, so the mean subtracted from the training rows was computed with the test rows included. Only the bottom row survives cross-validation honestly, and nothing about the top row raises an error.

The upper part shows:

scale everything → split → fit → evaluate.

It is wrong.

The lower part:

split → [fit scaler on train → fit model] → evaluate

is correct.

The most dangerous thing is that both versions **run without errors**.

Python does not know you have contaminated the test set.

Scikit-learn cannot know that your methodology was wrong.

The result will simply be a little more favourable than it should be.

It is the perfect example of the initial distinction:

working program ≠ valid experiment.

Slide 51 — Step 6: the result

Baseline: 0.629

Model: 0.986.

The natural temptation is to conclude:

excellent model.

But then comes:

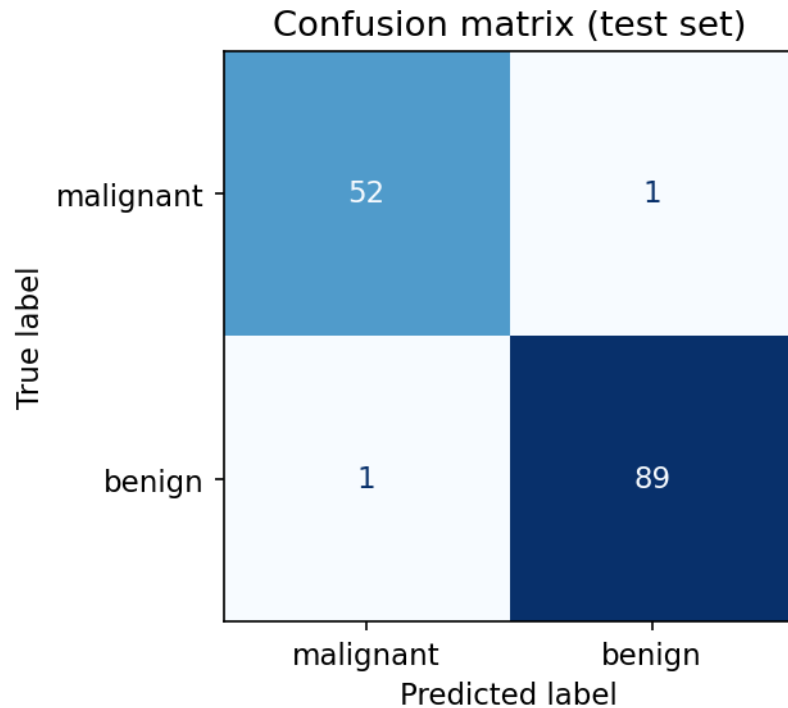
Done?

No.

A single number does not say **which** errors were made.

And that is exactly what the next slide will show.

Slide 52 — What accuracy hides



What a single accuracy figure hides: four outcomes collapsed into one number. Lesson 4 takes this apart properly.

The confusion matrix is:

	$\hat{y} = M$	$\hat{y} = B$
$y = M$	52	1
$y = B$	1	89

Total: $52 + 1 + 1 + 89 = 143$.

Errors: 2.

Accuracy:

$$\frac{141}{143} = 0.9860.$$

But one of the two errors is: $FN = 1$.

That is, a malignant tumour classified as benign.

The slide wants to make exactly this mental step:

0.986 looks almost perfect.

but:

“a malignant patient sent home” does not look almost perfect at all.

It is the contrast between **statistical representation** and **meaning in the application**.

Slide 53 — Precision and recall

For the malignant class:

Recall

$$Recall = \frac{TP}{TP + FN}.$$

With the values above:

$$Recall = \frac{52}{52 + 1} = \frac{52}{53} \approx 0.981.$$

Interpretation:

of the tumours that really were malignant, how many did we find?

Precision

$$Precision = \frac{TP}{TP + FP}.$$

So:

$$Precision = \frac{52}{52 + 1} = 0.981$$

which in this particular case is the same.

Interpretation:

of the tumours we flagged as malignant, how many really were?

The fact that precision and recall coincide here is accidental: it comes from: $FP = FN = 1$.

Normally they differ.

Trade-off By lowering the threshold, we predict more cases as positive: $Recall \uparrow$

but normally: $Precision \downarrow$.

Raising the threshold does the opposite.

Slide 54 — Precision and recall, counted

The previous slide's definitions as formulas, on the confusion matrix's own counts. Malignant is the positive class: **true positives** (TP), malignant and flagged, 52; **false negatives** (FN), malignant and missed, 1; **false positives** (FP), benign and flagged, 1. Then

$$\text{recall} = \frac{TP}{TP + FN} = \frac{52}{53} = 0.981, \quad \text{precision} = \frac{TP}{TP + FP} = \frac{52}{53} = 0.981.$$

Same numerator, different denominator: recall divides by everything that really was malignant, a row of the matrix; precision by everything that was flagged, a column. That is the whole difference, and the one most often mixed up.

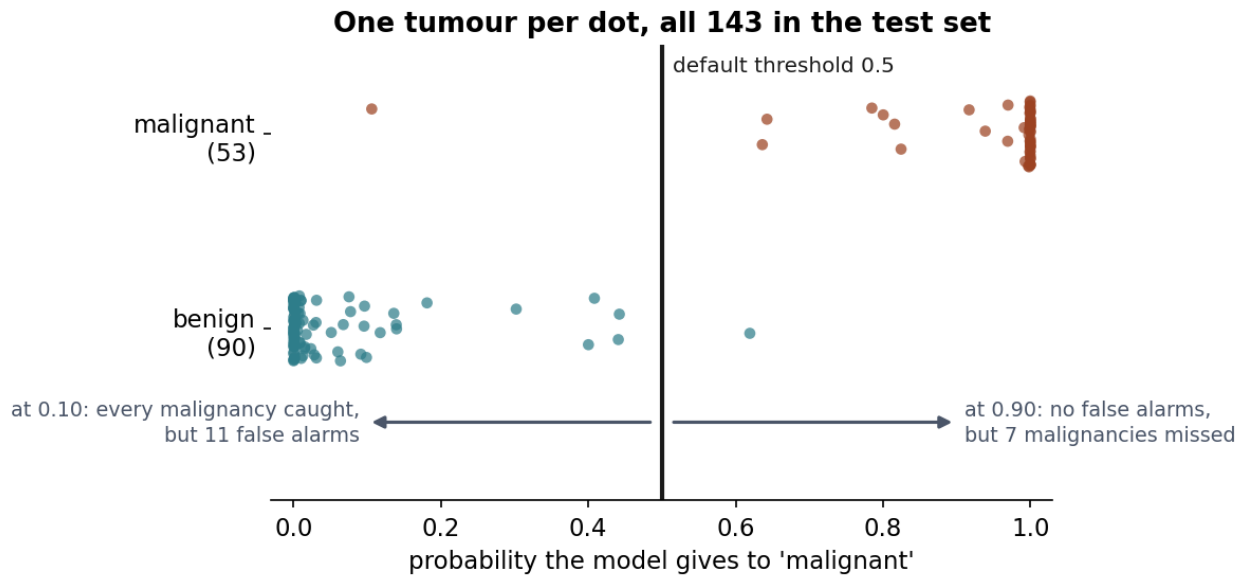
The two values are equal by coincidence: one miss and one false alarm, so both denominators are 53. The next slide shows where those labels come from, and that they can be moved, at which point the two numbers part.

Slide 55 — Behind every label, a probability

The step the confusion matrix hid. The model does not answer “malignant” or “benign”: it answers with **a probability** that the tumour is malignant (`predict_proba` in scikit-learn), and `predict()` is that number compared with **0.5**. The two errors just counted were not made by a confused model: the missed malignancy got 0.11, so the comparison with 0.5 called it benign; the false alarm was a benign tumour at 0.62. Where the probability comes from is lesson 4’s subject.

Then the question that opens the next slide: if a label is a comparison with 0.5, what happens with another threshold? At 0.10 the missed malignancy is caught, recall $53/53 = 1.000$, but eleven benign tumours join the alerts and precision falls to $53/64 = 0.828$. Nothing about the model changed; only the line moved. **0.5 is a default; nobody chose it.**

Slide 56 — The trade-off, drawn



One dot per tumour, on two rows: the 53 malignant cases above, the 90 benign below. Left to right is the probability the model gave to “malignant”, and the black line is the 0.5 default — everything to its right is called malignant, everything to its left benign.

This figure makes the threshold extremely concrete.

The model produces a probability: $p(y = \text{malignant} \mid x)$.

The standard rule is:

$$\hat{y} = \begin{cases} \text{malignant} & p \geq 0.5 \\ \text{benign} & p < 0.5. \end{cases}$$

But **0.5 is not a law of nature**.

It is simply the default.

If we lower it to: $t = 0.10$,

we catch every malignancy, but we get 11 false alarms.

If we raise it to: $t = 0.90$,

we eliminate the false alarms, but we miss 7 malignancies.

And the point that matters most is:

the model has not changed.

They are identical: w, b .

Only the decision rule has changed: $p \geq t$.

So there is no single answer to:

“how good is this classifier?”

until we specify the relative cost of the errors.

Slide 57 — What we did not do

So far we have not done:

tuning,
model comparison,
stability analysis.

And above all:

we have not looked at the test set to take decisions.

This is what makes the value interpretable: 0.986.

If, after seeing it, we had changed the model and recomputed, the test set would in effect have become a validation set.

Repeating this enough times, we would have started overfitting the test set as well.

This is a point that will be formalised in lesson 5.

Slide 58 — Four ways a model misleads you

The four categories are:

1. leakage,
2. imbalance,
3. shortcut features,
4. single-split noise.

What they have in common matters more than their differences:

all of them can produce a perfectly plausible number.

No crash.

No `ValueError`.

No absurd 500% accuracy.

The problem is discovered only by understanding how the number was produced.

Slide 59 — Notebook 3, live

The slide's body is a **toolbox table** pairing each failure with the calls that produce it and the slides that explain it: `SelectKBest(f_classif, k=20)` outside the pipeline and then inside it for leakage; `make_classification`, `accuracy_score` against `recall_score` and `confusion_matrix` for imbalance; `load_breast_cancer` and `SimpleImputer` for the shortcut column; `train_test_split(random_state=seed)` against `cross_val_score` for single-split noise.

Know what to watch for before you start, because the notebook is deliberately undramatic: **every cell runs, nothing raises, and every number it prints would pass a review.** The work is reading them. The two cells nobody should skip: the leaked selection reaching **77% on labels generated by a coin flip**, and the four deployment scenarios for the shortcut column, where the two that raise an exception turn out to be the lucky ones.

Slide 60 — 200 samples. 5000 random features. Coin-flip labels.

This is a very elegant controlled experiment.

We have: $m = 200$

and: $n = 5000$.

All the features are noise.

The labels are coin flips too: $P(y = 0) = P(y = 1) = 0.5$.

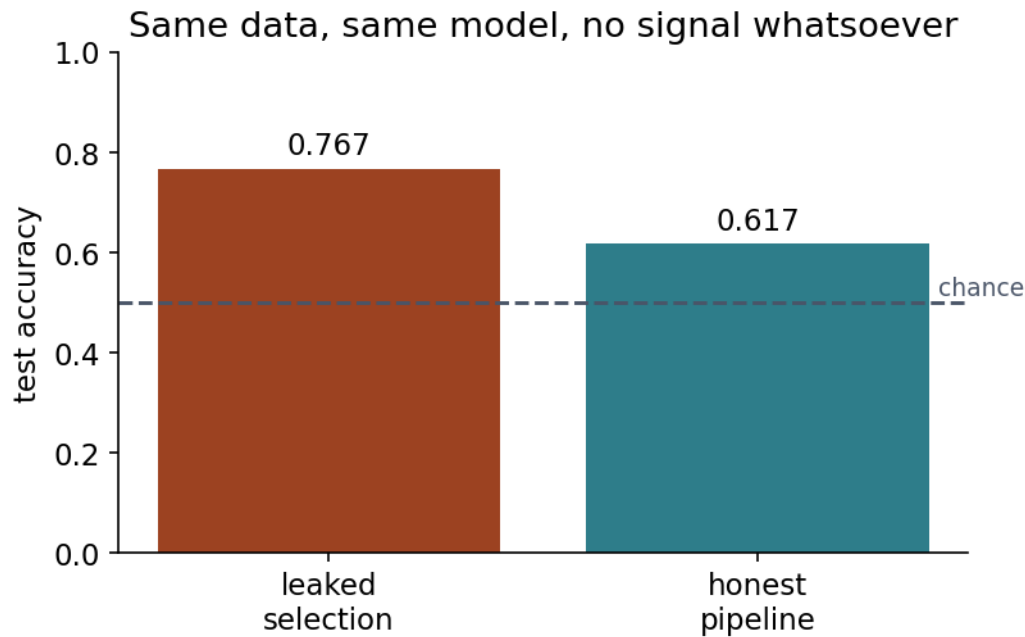
By construction: $X \perp Y$.

So there is no predictive signal.

The best possible classifier, in expectation, has: $accuracy = 0.5$.

Any systematically higher accuracy must therefore come from the experimental procedure.

Slide 61 — Select features first, split second



Selecting features before splitting: the test rows have already influenced which features exist, so the score that follows is not a measurement of anything.

This is a very powerful example of leakage.

Among 5000 random features, some will, by pure chance, have a relatively strong correlation with y .

Suppose we choose: 20

features showing the strongest association with the target **using the whole dataset**.

The problem is that we have also looked at: y_{test} .

So the features were chosen partly because they happen to predict exactly the labels we will later call “test”.

The leaked result is about: 0.767.

The honest comparison shown is: 0.617.

An important precision about the 0.617 Since the labels are random, the expected theoretical value is still: 0.5.

The 0.617 of this single run is therefore **sampling noise**: it is one particular, favourable test split.

It must not be read as “the correct pipeline really reaches 61.7%”.

Over many independent experiments it should return on average towards: 50%.

And it is almost ironic that this very figure anticipates the lesson on single-split noise.

Why do more features make the problem worse? Because the number of opportunities to find spurious correlations grows.

It is a multiple-comparisons effect.

With: $n = 20000$

there is a greater chance that some random features turn out exceptionally associated with y .

Slide 62 — The rule

Every step that learns anything must be fitted inside the training fold.

This is perhaps the most important practical rule of the lesson.

It does not apply only to the model.

It also applies to:

scaling, imputation, encoding, feature selection, tuning.

A correct way of thinking about the pipeline is:

everything whose parameters depend on data

must be treated as part of learning.

If a `fit()` appears anywhere, conceptually, it should not see the test fold.

Slide 63 — 99% accuracy, detecting nothing

Imagine: $P(y = 1) = 0.01$.

A classifier that always answers: $\hat{y} = 0$

gets about: *accuracy* = 99%.

But:

$$Recall_{positive} = \frac{TP}{TP + FN} = 0.$$

So the system finds **not even one** of the cases we care about.

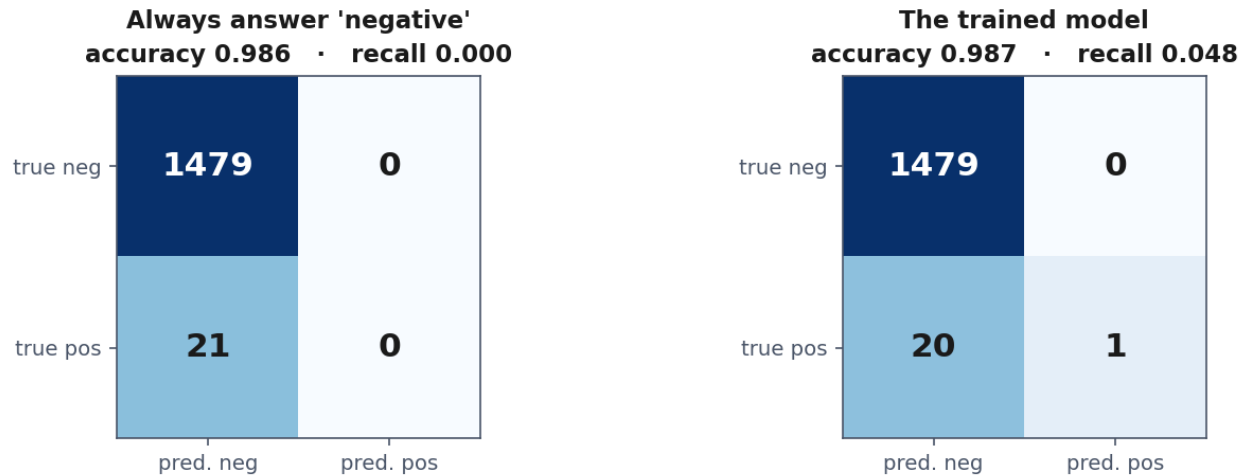
It is the simplest demonstration that a metric can be mathematically correct and useless in the application.

Under extreme class imbalance: *accuracy*

is dominated by the majority class.

Slide 64 — Two models, one accuracy

Nearly identical accuracy. One of them finds nothing.



99% accurate and detecting nothing. On a rare-event problem, accuracy measures the majority class and little else.

First classifier:

$$TN = 1479, \quad FP = 0, \quad FN = 21, \quad TP = 0.$$

Accuracy:

$$\frac{1479}{1500} = 0.986.$$

Recall: 0.

Second:

$$TN = 1479, \quad FP = 0, \quad FN = 20, \quad TP = 1.$$

Accuracy:

$$\frac{1480}{1500} = 0.98667 \approx 0.987.$$

Recall:

$$\frac{1}{21} \approx 0.0476.$$

So two models with practically indistinguishable accuracy:

0.986 vs 0.987

still behave differently on the problem that really matters.

But it is just as important to note that the second is practically useless too: it finds only one of the 21 positives.

So the slide shows that it is not enough to ask:

“what is the accuracy?”

We must ask:

“where are the errors?”

Slide 65 — Shortcut features

Example:

`biopsy_scheduled`

The feature is highly correlated with malignancy.

So it could raise accuracy a lot.

But it is recorded **after** the diagnostic suspicion has already been formed.

So at the moment we want to make the prediction: $x_{\text{biopsy_scheduled}}$
does not exist yet.

Depending on the situation, this is also called:

target leakage,
temporal leakage,
post-outcome feature.

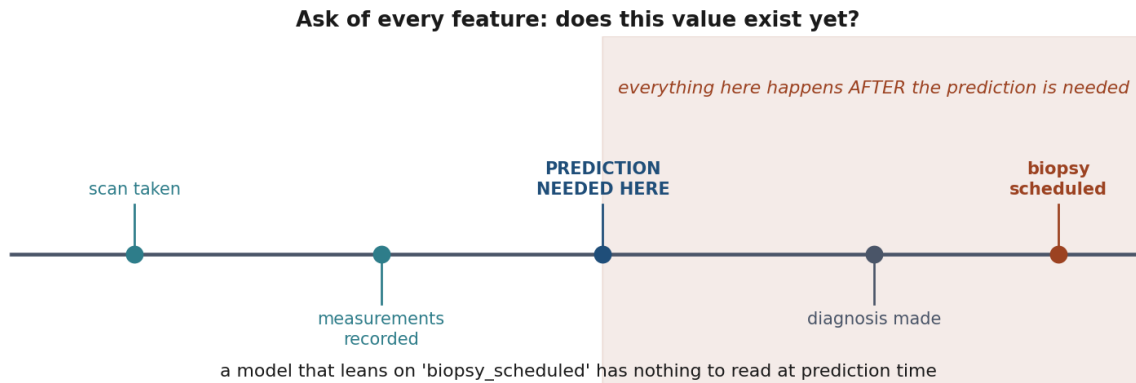
The difference from the previous leakage is crucial.

Here even a perfectly implemented cross-validation could produce excellent results.

Because statistically that feature really is predictive in the dataset.

The problem is **causal/temporal**, not statistical.

Slide 66 — Does this value exist yet?



A shortcut feature — something that predicts the label in this dataset for a reason that will not survive contact with new data. Everything to the right of the marked moment happens after a prediction is needed, so none of it can be an input.

The timeline makes the problem beautifully clear.

Before the moment of prediction:

scan taken → measurements recorded

so these features are legitimate.

After:

diagnosis made → biopsy scheduled

so they cannot be used for a prediction that should happen before the diagnosis.

The right question for every feature becomes:

Does this value exist yet?

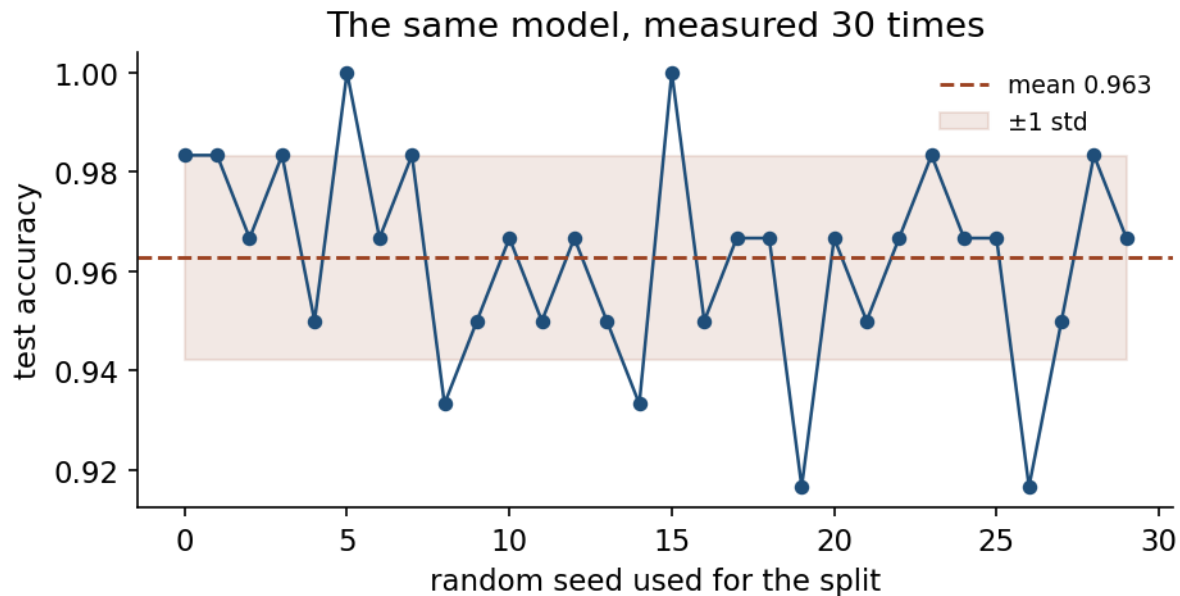
Not:

“does this feature correlate with the target?”

On the contrary, leaking features often correlate **wonderfully** with the target.

That is precisely why they are dangerous.

Slide 67 — One split is one measurement



One split is one measurement. Repeat it with a different seed and the number moves — which is why lesson 5 is about measuring properly.

Same model.

Same data.

We change only the random seed of the split.

Observed accuracy: $0.917 \rightarrow 1.000$.

Spread:

$$1.000 - 0.917 = 0.083.$$

That is, **8.3 percentage points**.

This means that if we compared two algorithms and found: $A = 0.96$

$B = 0.97$,

that percentage point could easily be much smaller than the noise introduced by the choice of split.

Cross-validation instead produces something like: 0.960 ± 0.030 .

The second number is almost as important as the first.

Saying: 0.960

without giving an idea of the spread leaves out essential information.

The slide leads directly into lesson 5.

Slide 68 — What the four have in common

The conclusion is:

None is a coding error.

So the four cases call for methodological questions.

The notes propose three excellent questions:

Where did this number come from?

What does it leave out?

Would it still hold on data I have never touched?

They are practically a universal checklist for reading an ML result.

We can sum them up formally as:

provenance + meaning + generalisation

Slide 69 — A model is not objective

Three strong but technically justified statements.

1. The model summarises the training data If the training data contains systematically distorted associations, ERM has no magic mechanism for recognising them as “unjust”.

It is simply trying to minimise: $\hat{R}(f)$.

2. Correlation is not causation A predictive model looks for: $P(Y | X)$,
or in any case a useful relationship between X and Y .

It does not automatically identify: $P(Y | do(X))$.

This distinction belongs to causal inference.

So a feature can be excellent predictively and entirely non-causal.

3. Accuracy is not the only property A real system may also require:

interpretability,
auditability,
robustness,
the possibility of review,
the cost of errors,
latency,
fairness.

These requirements must enter at the framing stage, not be added afterwards.

Slide 70 — Homework: we discuss it on Friday 9 October

The exercise uses wine quality.

But the interesting part is:

justify every decision.

So getting the best score is not what counts.

The **quality** variable takes discrete values 3–9.

A student could treat it as regression: $y \in \{3, \dots, 9\} \subset \mathbb{R}$

or build a classification, for example:

$$y = \begin{cases} 1 & \text{quality} \geq q \\ 0 & \text{quality} < q. \end{cases}$$

There is no automatically correct choice without specifying the use.

So the task is a check on **framing**, not only on implementation.

The task with ten random seeds is also interesting: it makes you see for yourself that a score is not an immutable value attached to the model.

Slide 71 — Before next week

The final slide sums up the working cycle:

environment → notebook → handout → quiz → exercise.

Lesson 2 will go deeper into data exploration, preprocessing and above all leakage.

The close is consistent with everything before it: the first lesson does not try to teach many algorithms.

Instead it wants to install the **right way of thinking about an ML experiment** before the actual algorithms arrive.

The thread of the whole lesson

The whole of L1 can be read as a very precise chain.

We start from a problem in which we cannot write the rule: rules impossible to specify.

Machine learning overturns the paradigm:

examples $\rightarrow$ learned function.

Formally we have: $(x, y) \sim \mathcal{D}$,

we look for: $f : \mathcal{X} \rightarrow \mathcal{Y}$,

and what we would like to minimise is:

$$R(f) = \mathbb{E}_{\mathcal{D}}[L(f(x), y)].$$

But $\mathcal{D}$ is unknown, so we cannot compute $R(f)$.

We can only compute:

$$\hat{R}_S(f) = \frac{1}{m} \sum_{i=1}^m L(f(x_i), y_i).$$

And the fundamental problem emerges:

$$\boxed{\hat{R}_S(f) \neq R(f)}$$

The gap between the two is the origin of overfitting.

The methodological answer is then:

Keep independent data to estimate generalisation.

from which practically the whole workflow follows:

Frame $\rightarrow$ Inspect $\rightarrow$ Split $\rightarrow$ Baseline $\rightarrow$ Pipeline $\rightarrow$ Evaluate $\rightarrow$ Diagnose $\rightarrow$ Iterate.

But not even a correct test set is enough if we measure the wrong thing.

Hence the confusion matrix, precision, recall and threshold:

metric $\neq$ application objective.

Finally the lesson shows four ways a plausible number can mislead us:

leakage, imbalance, shortcut, sampling noise.

The real conclusion of L1 is therefore much more general than “how to build a classifier”:

The central problem of machine learning is not obtaining a number, but understanding when that number deserves to be believed.

And it prepares directly for everything that follows: data preparation in L2, optimisation and regression in L3, classification and metrics in L4 and above all experimental methodology in L5.